

UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA

Dipartimento di Informatica, Sistemistica e Comunicazione (DISCo)

Corso di laurea in Data Science

Studio di robustezza dei modelli di machine learning al rumore nei dati

Progetto di *Architetture Dati*

Un'analisi sperimentale sul dataset MAGIC Gamma Telescope

Autori

Daniele Maccagnan – matricola [matricola]

Matias Bonoli – matricola [matricola]

Ashley Chudory – matricola [matricola]

Anno accademico 2025/2026

Indice

1	Introduzione	2
2	Il dataset	2
3	Suddivisione in training, validation e test	3
4	Analisi esplorativa dei dati	3
5	Preprocessing: standardizzazione	7
6	Modelli	7
7	Baseline su dati puliti	8
8	Metodologia di iniezione del rumore	11
8.1	Protocollo comune e livelli di intensità	11
8.2	I sette meccanismi	11
9	Risultati per tipo di rumore	12
9.1	Feature noise	12
9.2	Feature noise localizzato	15
9.3	Label noise random	18
9.4	Label noise borderline	21
9.5	Missing values	24
9.6	Outlier	26
9.7	Missing feature (sensore guasto)	28
10	Sintesi trasversale	31
10.1	Controllo multi-seed	34
11	Analisi non supervisionata (K-Means)	34
12	Conclusioni	36

1 Introduzione

In qualunque pipeline reale i dati non sono perfetti. Misurazioni imprecise, errori di annotazione, valori mancanti e anomalie sono la norma. Per questo non ci chiediamo quale modello raggiunga l'accuratezza più alta sui dati puliti, ma quanto ciascun modello resista quando i dati si degradano. Misuriamo la robustezza di tre modelli supervisionati corrompendo il training set in modo progressivo, con sette tipi di rumore.

Non vogliamo dichiarare un vincitore, ma capire come e perché ogni modello si degrada secondo il tipo di disturbo. Il protocollo è uniforme. Fissiamo un riferimento sui dati puliti, poi riaddestriamo gli stessi modelli su versioni del training set corrotte a tre livelli, 10%, 25% e 40%. Le metriche sono sempre calcolate sullo stesso test set pulito. I tre modelli sono una rete neurale semplice (*NN Base*), una sua versione regolarizzata (*NN Pro*) e un *Random Forest*. I sette rumori agiscono su feature, etichette, valori mancanti, outlier e disponibilità delle feature. Aggiungiamo un esperimento non supervisionato con K-Means, per vedere come agisce il rumore quando le etichette non entrano nel modello.

La relazione segue il flusso del lavoro. Descriviamo prima il dataset, la suddivisione dei dati e l'analisi esplorativa, condotta solo sul training set. Spieghiamo poi il preprocessing, i tre modelli e il loro comportamento sui dati puliti. Definiamo quindi i sette meccanismi di rumore e ne analizziamo gli effetti uno alla volta. Chiudiamo con una sintesi trasversale, l'esperimento non supervisionato e le conclusioni. Tutti i numeri e le figure vengono dall'esecuzione del notebook. Non abbiamo riaddestrato i modelli né ricalcolato le metriche, così il testo resta coerente con il codice. Per la riproducibilità abbiamo fissato un seme casuale comune (`seed = 42`) su tutte le componenti stocastiche, cioè suddivisione dei dati, inizializzazione delle reti, iniezione del rumore, Random Forest e K-Means.

2 Il dataset

Il dataset è il MAGIC Gamma Telescope, generato da simulazioni Monte Carlo del telescopio Cherenkov MAGIC. Ogni riga descrive la traccia luminosa, di forma circa ellittica, che un evento atmosferico lascia sul piano del rivelatore. L'obiettivo è distinguere i segnali utili, prodotti da raggi gamma, dal fondo di adroni, cioè raggi cosmici. È una classificazione binaria con un significato fisico. Un adrone classificato come gamma è un falso positivo e contamina il segnale. Un gamma scartato è un falso negativo e riduce la statistica utile. I due errori hanno costi diversi, quindi la sola accuratezza non basta a giudicare un modello.

Il file di partenza contiene 19020 osservazioni e 11 colonne, dieci feature continue e una variabile target. Le feature riassumono la geometria e la distribuzione di intensità dell'immagine dell'evento. Abbiamo assegnato i nomi alle colonne e mappato le due classi in valori numerici, $h \rightarrow 0$ per gli adroni e $g \rightarrow 1$ per i gamma. La Tabella 1 riassume il significato delle dieci feature.

Tabella 1: Le dieci feature continue del dataset MAGIC e il loro significato.

Feature	Descrizione
fLength	lunghezza dell'asse maggiore dell'ellisse dell'immagine
fWidth	lunghezza dell'asse minore dell'ellisse
fSize	logaritmo della somma delle intensità di tutti i pixel
fConc	rapporto di concentrazione dei due pixel più intensi
fConc1	rapporto di concentrazione del pixel più intenso
fAsym	asimmetria della distribuzione lungo l'asse maggiore
fM3Long	momento terzo (asimmetria) lungo l'asse maggiore
fM3Trans	momento terzo lungo l'asse minore
fAlpha	angolo tra l'asse maggiore e la direzione del centro del campo
fDist	distanza dal centro del campo all'origine dell'ellisse
class	target: 1 = gamma (segnale), 0 = adrone (fondo)

3 Suddivisione in training, validation e test

Abbiamo suddiviso il dataset in tre parti prima di ogni altra operazione, inclusa l'analisi esplorativa. Separare gli insiemi fin dall'inizio evita che statistiche calcolate sull'intero dataset influenzino, anche in modo indiretto, le scelte successive. Sia l'analisi esplorativa sia le statistiche di preprocessing usano solo il training set, così validation e test restano una stima onesta della generalizzazione.

La suddivisione avviene in due passaggi, con proporzioni 60% / 20% / 20% per training, validation e test. Entrambi i passaggi sono stratificati sul target: ogni sottoinsieme mantiene le stesse proporzioni tra gamma e adroni del dataset completo. Serve perché le classi sono sbilanciate; senza stratificazione un sottoinsieme potrebbe contenere troppo pochi adroni. In valori assoluti otteniamo 11 412 campioni di training, 3 804 di validation e 3 804 di test. Il validation set serve a controllare l'addestramento delle reti, mentre il test resta intatto per la valutazione finale.

4 Analisi esplorativa dei dati

Conduciamo l'analisi esplorativa solo sul training set, per isolare validation e test come spiegato nella Sezione 3. Il dataset non contiene valori nulli. La condizione di partenza è pulita, quindi lo studio resta controllato, perché tutta la corruzione la introduciamo noi in modo deliberato e quantificabile. Abbiamo poi rimosso 41 righe duplicate, portando il training set da 11 412 a 11 371 osservazioni. Eliminare i duplicati evita che campioni identici pesino più del dovuto in alcune zone dello spazio delle feature.

La Figura 1 mostra la distribuzione del target ed evidenzia uno sbilanciamento. I gamma sono circa il 64,8% delle osservazioni, gli adroni il restante 35,2%, quasi due a uno. Lo squilibrio ha una conseguenza pratica. Un classificatore banale che predice sempre la classe maggioritaria raggiunge già un'accuratezza vicina al 65%. Sotto questa soglia un risultato non dice nulla, perché non batte la previsione costante.

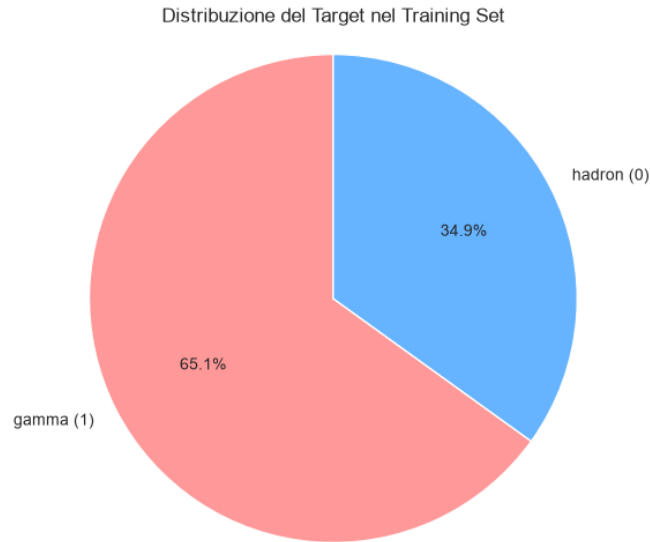


Figura 1: Distribuzione delle due classi nel training set: i gamma (segnale) sono quasi il doppio degli adroni (fondo).

Le distribuzioni delle feature (Figura 2) sono quasi tutte asimmetriche, concentrate sui valori bassi e con code lunghe verso destra. L'asimmetria conta. La mediana è meno sensibile ai valori estremi e descrive il centro di queste distribuzioni meglio della media. Le feature occupano anche intervalli numerici molto diversi, da poche unità a qualche centinaio.

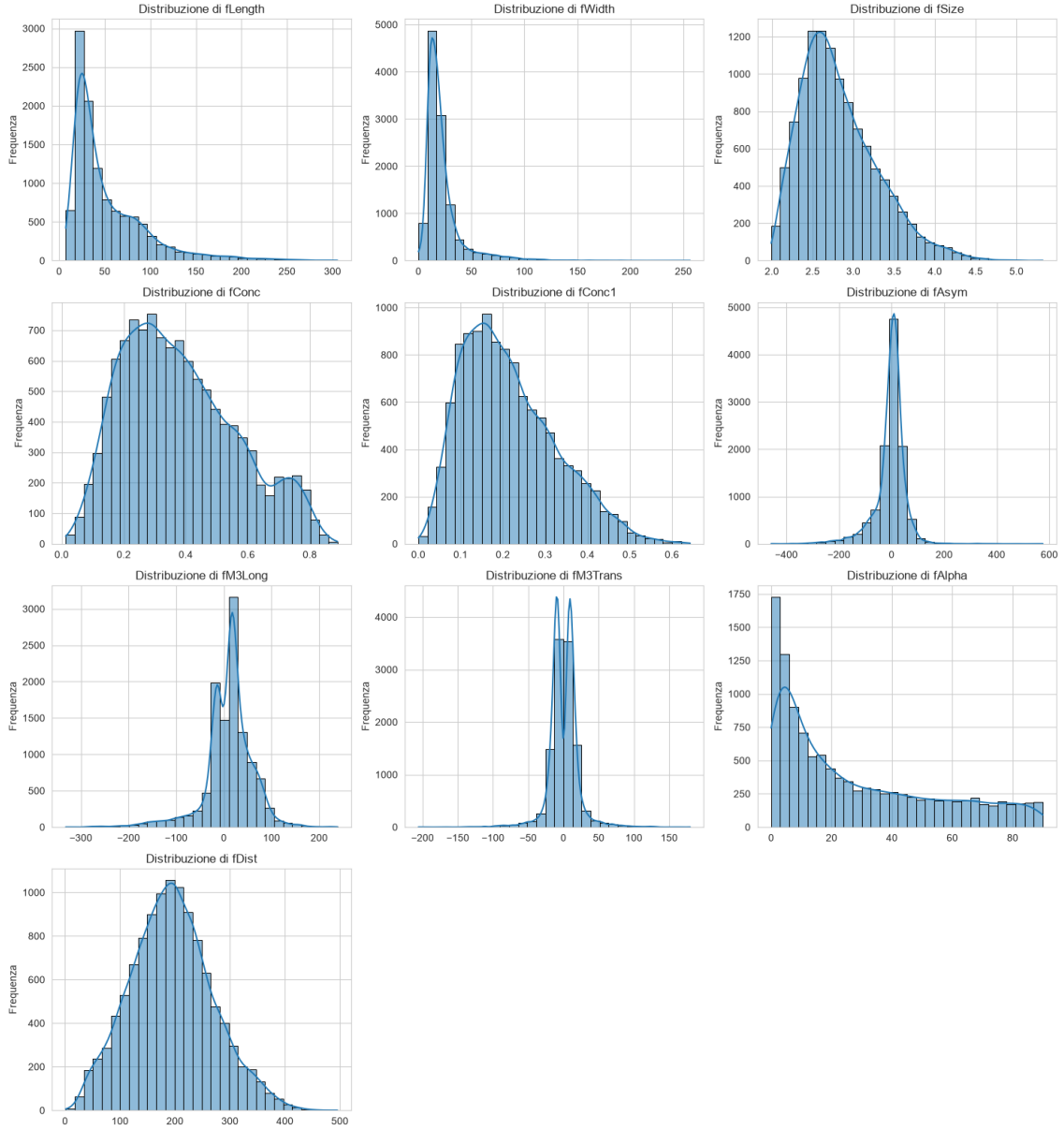


Figura 2: Istogrammi delle dieci feature continue (training set): distribuzioni asimmetriche, con code lunghe e scale eterogenee.

I boxplot per classe (Figura 3) mostrano quali feature separano gamma e adroni. La differenza più netta è su **fAlpha**. Per gli adroni copre un ampio intervallo, per i gamma si concentra su valori piccoli. Seguono **fLength**, **fWidth** e **fSize**, con un buon potere discriminante. **fM3Trans** e **fAsym** hanno invece distribuzioni quasi sovrapposte tra le due classi e separano poco. L'informazione utile non è distribuita in modo uniforme, ma concentrata in poche feature.

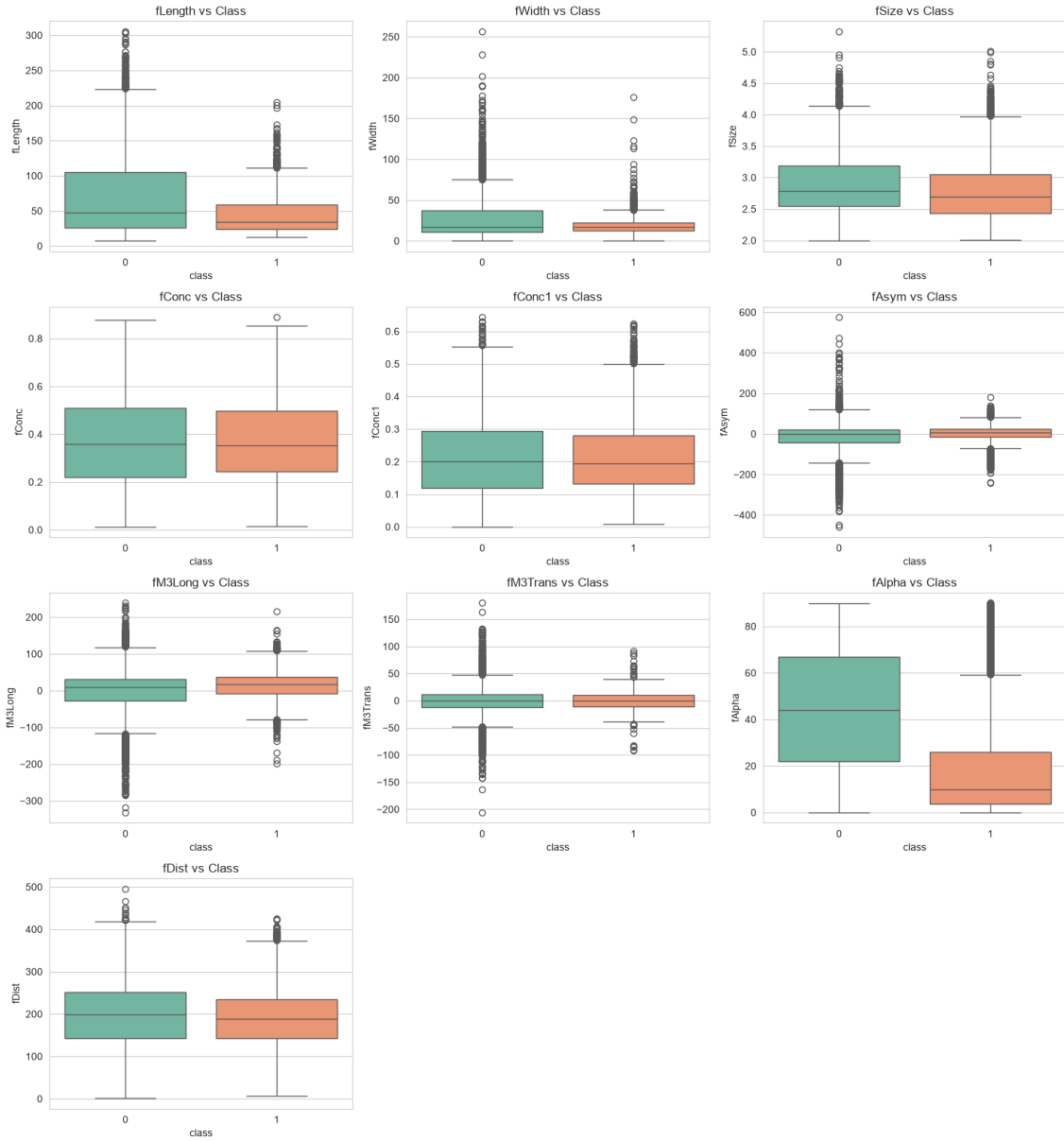


Figura 3: Boxplot di ogni feature per classe (training set): **fAlpha** è la più discriminante, i momenti trasversali separano poco le classi.

La matrice di correlazione (Figura 4) mostra alcune relazioni lineari forti. **fConc** e **fConc1** sono quasi ridondanti, con correlazione 0,98, e **fSize** è anticorrelata con entrambe, $-0,85$ e $-0,81$. Anche la terna **fLength**, **fWidth**, **fSize** ha correlazioni elevate, tra 0,71 e 0,77. Queste feature trasportano informazione in parte sovrapposta, quindi il disturbo o la perdita di una può essere compensato dalle altre. **fAlpha**, al contrario, è la più discriminante ma quasi scorrelata da tutte, e la sua informazione è unica e non recuperabile altrove.

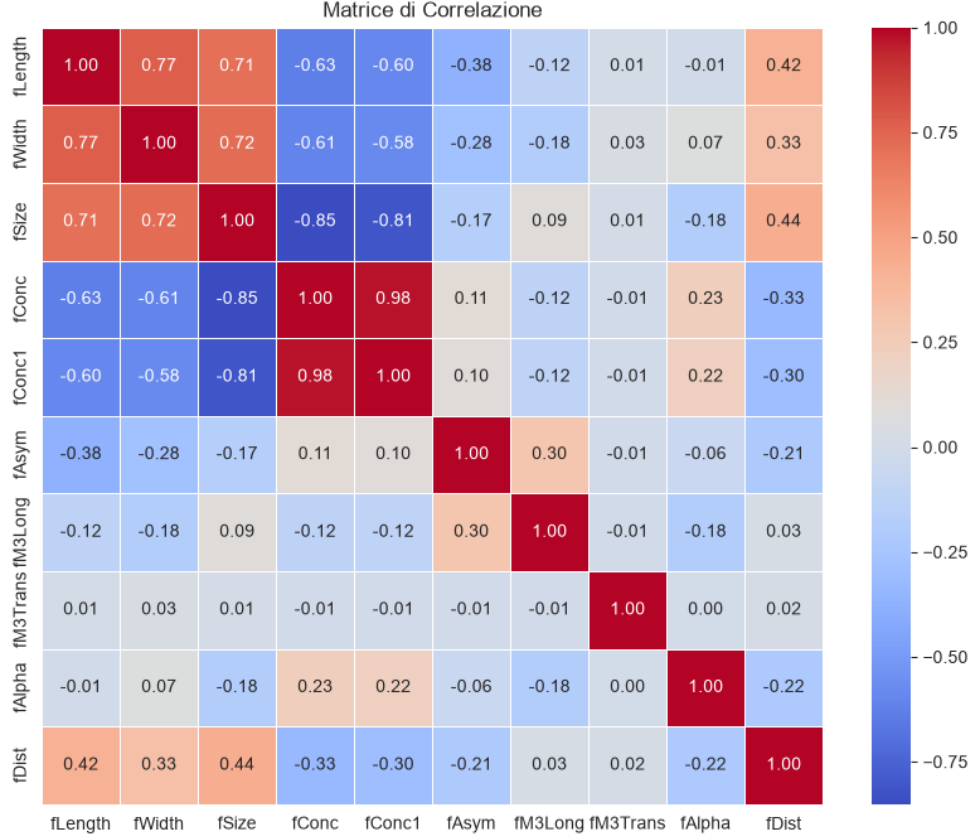


Figura 4: Matrice di correlazione tra le feature (training set). Spiccano la quasi-ridondanza `fConc`/`fConc1` e l'anticorrelazione di `fSize` con le concentrazioni; `fAlpha` è quasi scorrelata da tutte.

5 Preprocessing: standardizzazione

Nell'analisi esplorativa (Sezione 4) le feature mostravano scale numeriche molto diverse. Le abbiamo quindi standardizzate, portandole a media nulla e deviazione standard unitaria. Questo conta soprattutto per le reti neurali, che convergono meglio quando gli input hanno scale simili. Senza standardizzazione, le feature con valori più grandi dominerebbero il calcolo dei gradienti per la loro scala, non per la loro rilevanza.

Abbiamo calcolato lo standardizzatore solo sul training set e l'abbiamo poi applicato, con gli stessi parametri, a validation e test. Così il preprocessing non usa alcuna informazione da validation e test ed evita il *data leakage*, perché media e deviazione standard vengono solo dal training set.

6 Modelli

Confrontiamo tre modelli, scelti per rappresentare livelli crescenti di difesa contro il rumore. La Tabella 2 ne riassume i parametri principali.

La rete neurale Base è un percettore multistrato volutamente semplice. Prende in ingresso le dieci feature standardizzate e le elabora con due strati nascosti densi, il primo da 32 neuroni e il secondo da 16, entrambi con attivazione ReLU. Un neurone finale con attivazione sigmoide produce la probabilità che l'evento sia un gamma. L'addestramento

usa Adam con tasso di apprendimento 10^{-3} e perdita binary cross-entropy, per 75 epoche con batch da 32. La rete non ha regolarizzazione. È il modello “ingenuo”, quello che ci aspettiamo più esposto al rumore, perché nulla gli impedisce di adattarsi anche ai dati corrotti.

La rete neurale Pro ha la stessa architettura, due strati nascosti da 32 e 16 neuroni, ma aggiunge quattro difese contro l’overfitting. La regolarizzazione $L2$, con coefficiente 10^{-4} sui pesi dei due strati nascosti, penalizza i pesi grandi e scoraggia soluzioni troppo complesse. Il *dropout*, con probabilità 0,2 dopo ogni strato nascosto, disattiva a caso una frazione di neuroni a ogni passo, così la rete non dipende da singole unità e costruisce rappresentazioni più ridondanti. Il *class weight* bilanciato dà più peso agli adroni, la classe minoritaria, perché i loro errori non vengano trascurati a favore della classe abbondante. L’*early stopping* monitora la perdita sul validation set e ferma l’addestramento quando non migliora per 15 epoche consecutive, entro un massimo di 150, e ripristina i pesi migliori.

Il Random Forest è un insieme di 200 alberi decisionali, ciascuno addestrato su un sottoinsieme casuale di dati e feature. La robustezza nasce dalla media tra molti alberi, perché gli errori del singolo albero tendono a cancellarsi nell’aggregazione per voto. Non procede per epoche, quindi non ha curve di addestramento; ne mostriamo solo la matrice di confusione e il report di classificazione.

Tabella 2: Parametri principali dei tre modelli confrontati.

Modello	Architettura	Regolarizzazione / iperparametri
NN Base	Dense(32) → Dense(16) → sigmoide	nessuna; Adam 10^{-3} , 75 epoche, batch 32
NN Pro	Dense(32) → Dense(16) → sigmoide	$L2$ 10^{-4} , dropout 0,2, class weight bilanciato, early stopping (patience 15), max 150 epoche
Random Forest	200 alberi decisionali	aggregazione per voto, <code>random_state=42</code>

Usiamo quattro metriche, tutte calcolate sul test set pulito applicando alla probabilità prevista la soglia di 0,5. L’accuratezza è immediata ma poco affidabile con classi sbilanciate. L’F1 macro e la recall macro mediano le due classi con lo stesso peso, quindi penalizzano un modello che trascura la classe minoritaria. L’AUC misura la capacità di ordinare i campioni a prescindere dalla soglia.

7 Baseline su dati puliti

Prima di introdurre il rumore addestriamo i tre modelli sui dati puliti. Otteniamo così il riferimento a livello 0%, rispetto al quale misureremo ogni degrado. La Tabella 3 riassume le metriche di test.

Tabella 3: Metriche di test dei tre modelli sui dati puliti (riferimento a 0% di rumore).

Modello	Accuratezza	F1 macro	AUC
NN Base	0,8678	0,8517	0,9214
NN Pro	0,8601	0,8468	0,9238
Random Forest	0,8785	0,8629	0,9300

I tre modelli partono da prestazioni vicine, con accuratezza tra l'86 e l'88% e AUC tra 0,92 e 0,93. Il Random Forest è leggermente il migliore su tutte le metriche, mentre la rete Pro ha un'accuratezza appena inferiore alla Base. È un risultato atteso, perché la regolarizzazione cede un po' di adattamento ai dati puliti in cambio di stabilità. L'AUC della rete Pro è però già superiore a quella della Base, quindi l'ordinamento delle probabilità è più solido nonostante l'accuratezza più bassa.

Le matrici di confusione (Figura 5) mostrano il comportamento tipico con classi sbilanciate. Tutti riconoscono bene i gamma, la classe abbondante, e sbagliano di più sugli adroni. Le curve di addestramento delle due reti (Figura 6) convergono in modo regolare, con un divario contenuto tra train e validation, e sui dati puliti nessuna delle due va in overfitting.

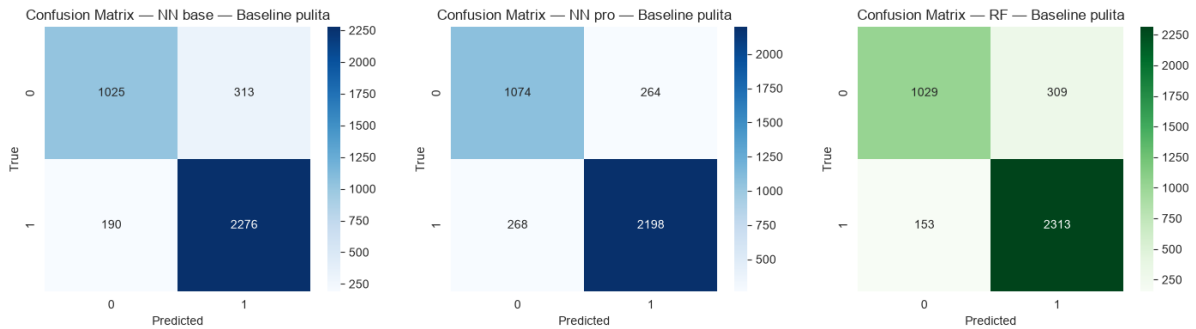


Figura 5: Matrici di confusione sui dati puliti: rete Base, rete Pro e Random Forest (da sinistra a destra). Tutti riconoscono bene i gamma, gli errori si concentrano sugli adroni.

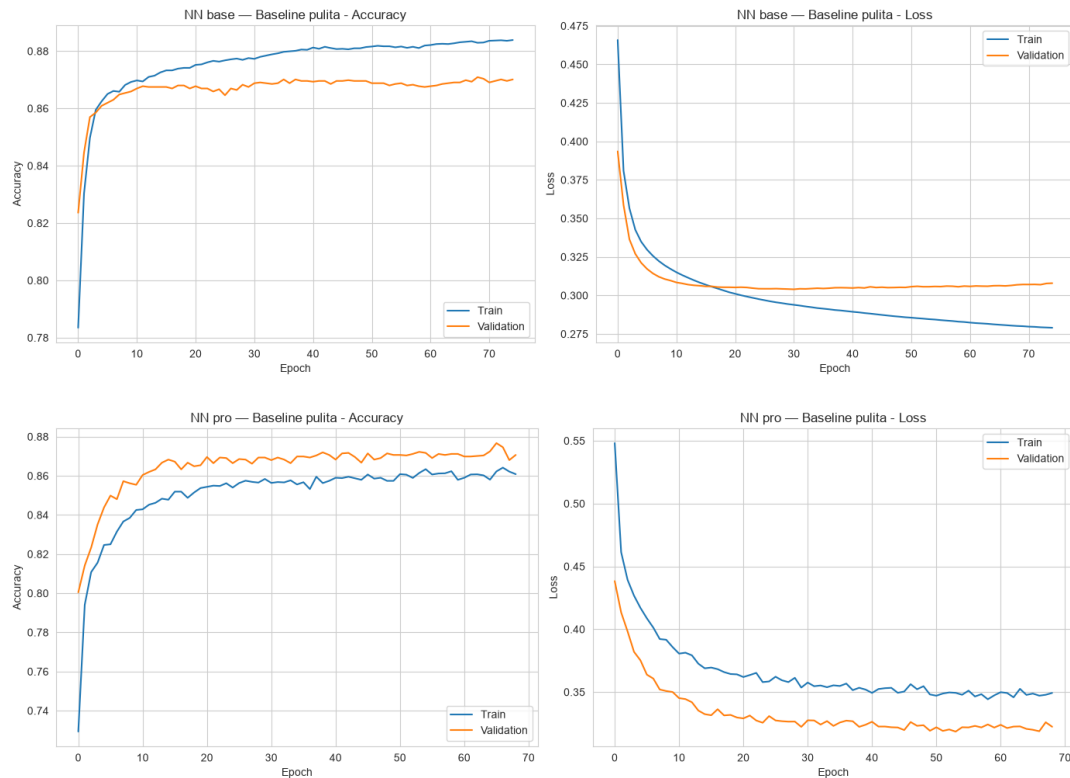


Figura 6: Curve di accuratezza e loss (train vs validation) sui dati puliti: rete Base (in alto) e rete Pro (in basso).

Calcoliamo poi l'importanza delle feature secondo il Random Forest (Figura 7). La gerarchia conferma con i numeri quanto si vedeva nei boxplot (Figura 3). **fAlpha** è di gran lunga la più importante, con un valore vicino a 0,24, più del doppio della seconda. Seguono **fLength**, **fWidth** e **fSize**.

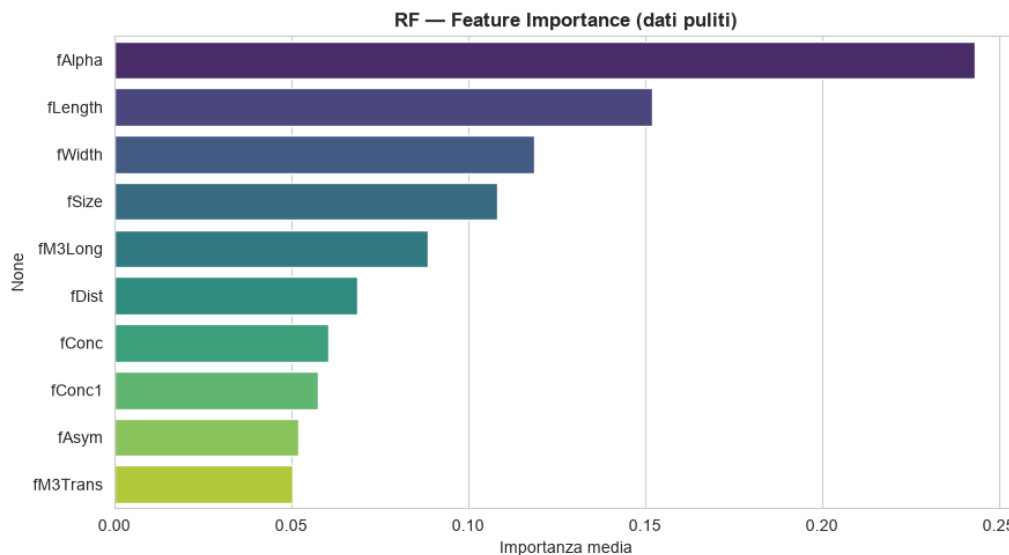


Figura 7: Importanza delle feature secondo il Random Forest sui dati puliti. **fAlpha** domina, coerentemente con la separabilità osservata nei boxplot (Figura 3).

8 Metodologia di iniezione del rumore

Definiamo sette tipi di rumore, ciascuno pensato per riprodurre un guasto realistico di una pipeline dati. La Tabella 4 li riassume.

Tabella 4: I sette tipi di rumore studiati e il guasto reale che simulano.

Tipo di rumore	Cosa simula
Feature noise	misurazioni imprecise (rumore gaussiano sulle feature)
Feature noise localizzato	misurazioni imprecise sui canali più informativi
Label noise random	errori di etichettatura casuali e uniformi
Label noise borderline	errori di etichettatura sui campioni ambigui
Missing values	celle mancanti (NaN) da imputare
Outlier	valori estremi anomali da trattare
Missing feature	uno o più sensori guasti (feature non disponibili)

8.1 Protocollo comune e livelli di intensità

In tutti gli esperimenti il rumore colpisce solo il training set, mentre validation e test restano puliti. Vogliamo misurare quanto un modello addestrato su dati corrotti riesca comunque a generalizzare su dati corretti. Per ogni tipo di rumore riaddestriamo tutti e tre i modelli sul riferimento pulito e sui tre livelli, calcolando ogni volta le metriche sullo stesso test set pulito.

Ogni rumore agisce a tre livelli, 10%, 25% e 40%. Cosa significhi il livello dipende dal disturbo, come spieghiamo di seguito meccanismo per meccanismo.

8.2 I sette meccanismi

Feature noise (rumore gaussiano). Aggiungiamo a ogni feature del training un rumore gaussiano a media nulla e deviazione standard pari al livello. I dati sono standardizzati (Sezione 5), quindi un livello di 0,40 corrisponde a un rumore ampio il 40% della deviazione standard della feature, uguale per tutte. Simula sensori imprecisi che restituiscono misure leggermente sbagliate.

Feature noise localizzato. Qui teniamo l'intensità del rumore gaussiano fissa a $\sigma = 0,35$ e facciamo crescere il numero di feature colpite, seguendo la gerarchia di importanza del Random Forest (Figura 7). Al 10% il rumore agisce sulla sola **fAlpha**, al 25% sulle prime tre, al 40% sulle prime cinque. A differenza del feature noise, che aumenta l'intensità su tutte le feature, qui colpiamo solo le più informative. Simula sensori imprecisi sui canali che pesano di più.

Label noise random. Invertiamo l'etichetta, da 0 a 1 e viceversa, di una frazione di campioni del training pari al livello, scelti a caso in modo uniforme. Simula errori di annotazione senza schema.

Label noise borderline. Invertiamo una frazione di etichette pari al livello, ma la selezione non è casuale. Calcoliamo i centroidi delle due classi nello spazio standardizzato e, per ogni campione, la distanza dal centroide della classe opposta. Invertiamo le

etichette dei campioni con distanza minore, cioè quelli più vicini all'altra classe e quindi più ambigui. È un disturbo più insidioso del rumore casuale, perché corrompe gli esempi vicini al confine decisionale, dove la separazione è più delicata. Ci aspettiamo quindi che, a parità di percentuale, danneggi più del rumore casuale.

Missing values. Sostituiamo ogni cella con un valore mancante (NaN) con probabilità pari al livello, in modo indipendente. Né le reti né il Random Forest accettano NaN, quindi serve un'imputazione; confrontiamo due strategie, la media e la mediana della feature. Coerentemente con il protocollo comune, il disturbo e l'imputazione agiscono solo sul training, mentre validation e test restano puliti. Come variante canonica per i confronti aggregati scegliamo la mediana, perché le distribuzioni sono asimmetriche (Figura 2) e la mediana è un riassunto più stabile della media.

Outlier. Scegliamo una frazione di campioni del training pari al livello e ne sostituiamo tutti i valori con quantità estreme, pari a ± 5 deviazioni standard con segno casuale per ogni cella. Confrontiamo tre risposte. La prima lascia gli outlier inalterati. La seconda li *winsorizza*, cioè tronca il 5% delle code di ogni feature per attenuare gli estremi senza eliminarli. La terza li *imputa*, cioè marca come mancanti i valori con punteggio z in modulo sopra 4 e li sostituisce con la mediana. Come variante canonica adottiamo la winsorizzazione, un buon compromesso tra semplicità ed efficacia.

Missing feature (sensore guasto). Azzeriamo una o più feature, come se i sensori che le producono si guastassero. Lo scenario non ha un livello percentuale naturale, quindi lo mappiamo sulla rimozione progressiva delle feature più importanti secondo il Random Forest (Figura 7). Al 10% togliamo solo `fAlpha`, al 25% le prime tre, al 40% le prime cinque. La rimozione vale per training, validation e test. Partire dalle feature più importanti rende lo scenario volutamente pessimistico, perché simuliamo il guasto dei sensori più importanti, non di quelli marginali.

9 Risultati per tipo di rumore

Esaminiamo i sette disturbi uno alla volta, per vedere come le prestazioni si degradano al crescere del rumore. Per ogni rumore riportiamo la tabella delle metriche ai quattro livelli e i grafici di confronto tra i tre modelli su accuratezza e F1 macro. Dove serve, aggiungiamo le curve di addestramento e le matrici di confusione nel caso più estremo. Il meccanismo di ogni disturbo è quello della Sezione 8.

9.1 Feature noise

Il rumore gaussiano sulle feature produce un degrado graduale e contenuto. La Tabella 5 mostra che anche al 40% tutti i modelli restano sopra l'83% di accuratezza, con un calo di tre-quattro punti rispetto al riferimento. Le curve di accuratezza (Figura 8, a sinistra) restano vicine e quasi piatte, e l'F1 macro (a destra) conferma lo stesso quadro.

Tabella 5: Feature noise: metriche al variare del livello di rumore (test set).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8657	0.85	0.9187	0.8636	0.85	0.9204	0.8646	0.84	0.9222
25%	0.8520	0.83	0.9074	0.8465	0.83	0.9071	0.8494	0.82	0.9041
40%	0.8328	0.80	0.8941	0.8417	0.82	0.8940	0.8318	0.80	0.8913

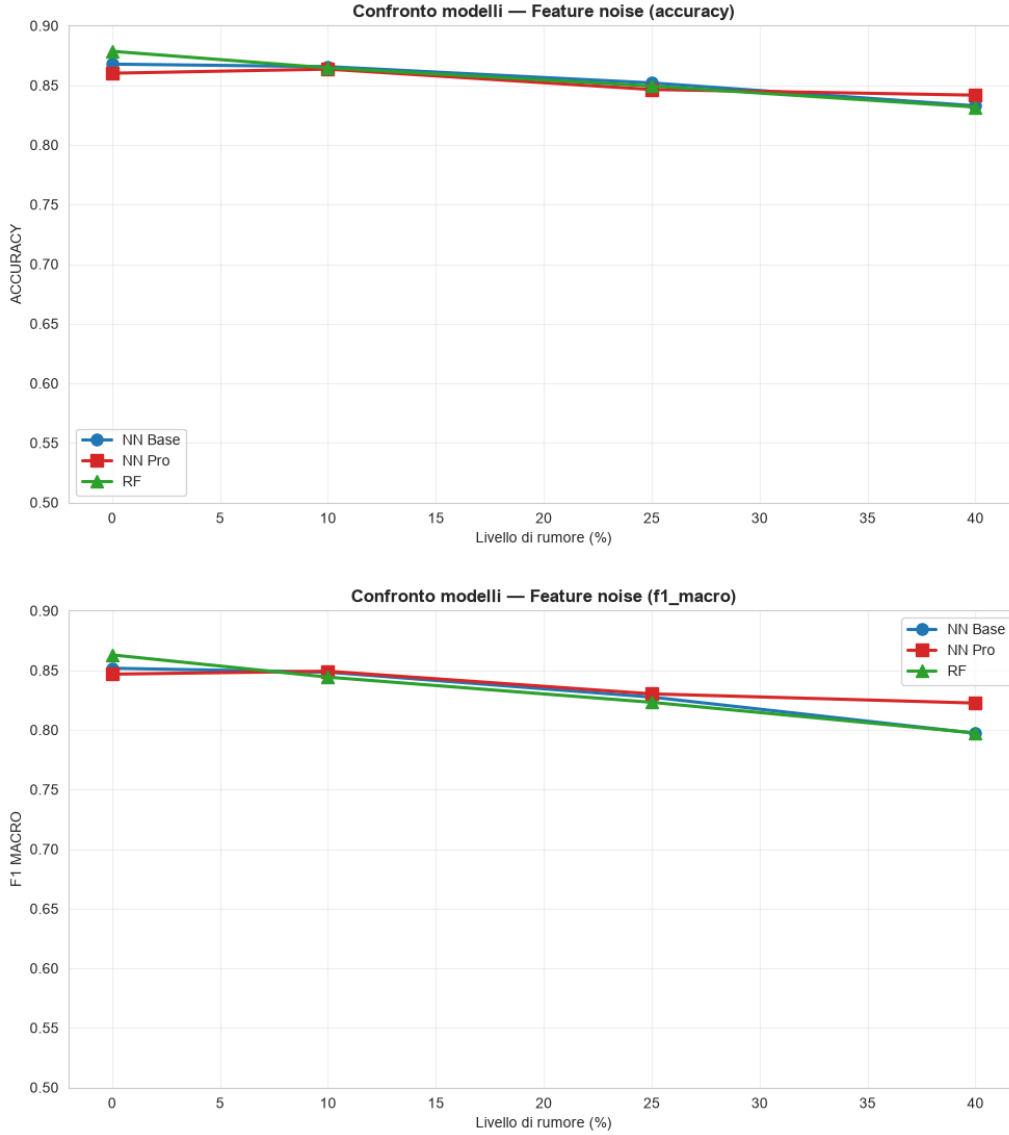


Figura 8: Feature noise: accuratezza (in alto) e F1 macro (in basso) dei tre modelli al crescere del rumore.

Al 40% la rete Pro (0,8417) supera sia la Base (0,8328) sia il Random Forest (0,8318). È la prima conferma della nostra ipotesi sulla regolarizzazione. Con feature rumorose, L2 e dropout impediscono alla rete di inseguire le fluttuazioni casuali del segnale e la tengono più stabile; la rete Base, senza freni, si adatta al rumore e perde terreno. Le curve di addestramento al 40% (Figura 9) mostrano la differenza. Nella rete Base resta un piccolo

divario tra train e validation, nella rete Pro le due curve sono quasi sovrapposte. Il calo resta modesto grazie alla ridondanza tra le feature (Figura 4), perché il rumore su una feature è in parte compensato dall'informazione correlata nelle altre.

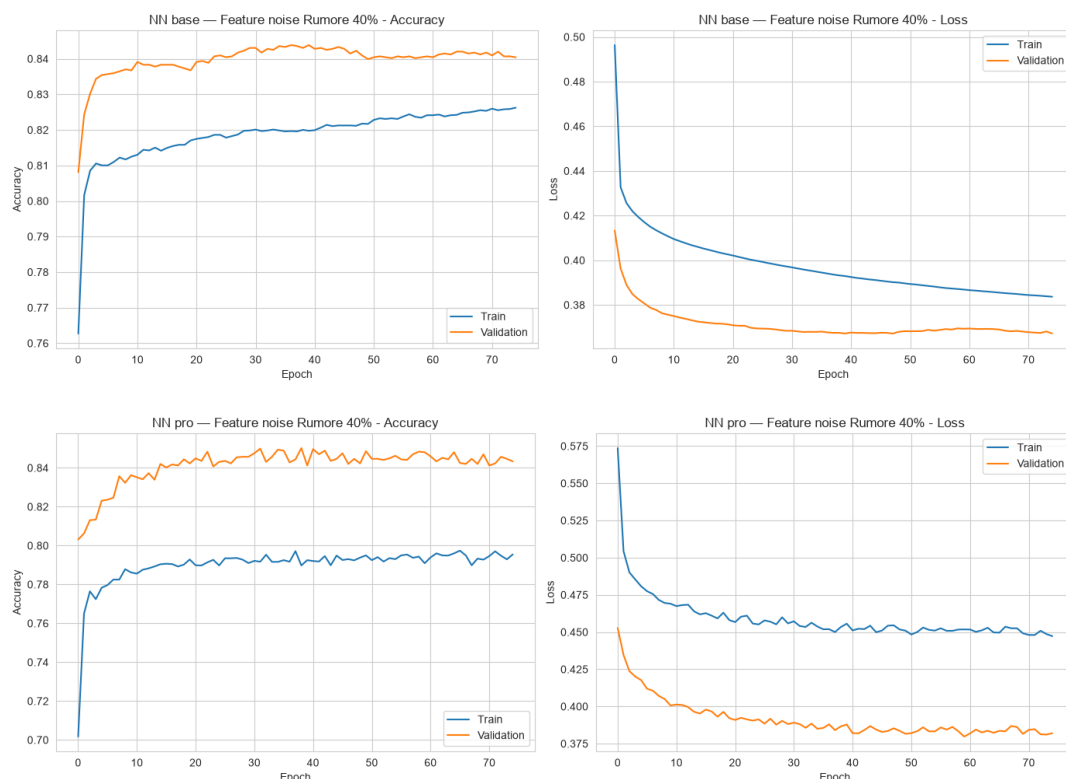


Figura 9: Feature noise al 40%: curve di addestramento della rete Base (in alto) e della rete Pro (in basso). Nella rete Pro train e validation restano sovrapposte: la regolarizzazione evita l'adattamento al rumore.

Il degrado, però, è asimmetrico tra le due classi. Il dataset è sbilanciato (Sezione 4), quindi l'accuratezza resta alta anche quando il modello inizia a sacrificare la classe minoritaria. Sotto feature noise al 40% la recall degli adroni della rete Base scende da 0,77 a 0,59, mentre quella dei gamma sale da 0,92 a 0,97, perché il modello sposta sempre più adroni verso la classe maggioritaria e li classifica come gamma. Qui pesa la *class weight* bilanciato della rete Pro (Sezione 6), che tiene la recall degli adroni a 0,73, contro lo 0,59 della Base e lo 0,60 del Random Forest. È questa protezione della classe rara a spiegare il suo F1 macro più alto a parità di accuratezza. Le matrici di confusione della rete Base (Figura 10) mostrano il fenomeno. Livello dopo livello cresce la colonna delle predizioni "gamma", perché sempre più adroni (riga 0) finiscono nel segnale, mentre i gamma (riga 1) restano riconosciuti.

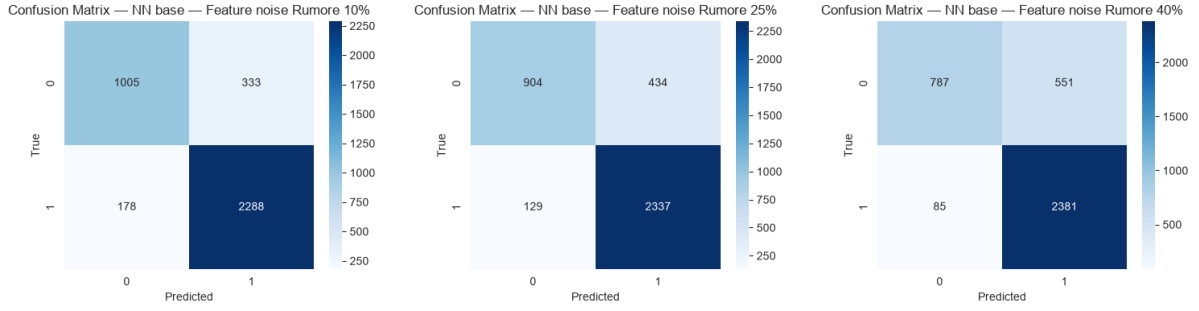


Figura 10: Feature noise, rete Base: matrici di confusione al 10%, 25% e 40% (da sinistra a destra). Gli adroni (riga 0) vengono classificati come gamma con frequenza crescente, mentre i gamma (riga 1) restano riconosciuti.

9.2 Feature noise localizzato

Concentriamo ora il rumore sulle feature più informative invece di spalmarlo su tutte. L'intensità resta fissa a $\sigma = 0,35$ e cresce il numero di feature disturbate, secondo il meccanismo della Sezione 8. Al 10% il rumore colpisce solo **fAlpha**, al 25% le prime tre, al 40% le prime cinque. La Tabella 6 raccoglie le metriche.

Tabella 6: Feature noise localizzato ($\sigma = 0,35$) sulle sole feature più importanti: il livello indica quante feature sono colpite (10%→top-1, 25%→top-3, 40%→top-5).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8651	0.85	0.9211	0.8651	0.85	0.9243	0.8743	0.86	0.9274
25%	0.8523	0.83	0.9043	0.8478	0.83	0.9058	0.8533	0.83	0.9056
40%	0.8452	0.82	0.8991	0.8386	0.82	0.8973	0.8389	0.81	0.8931

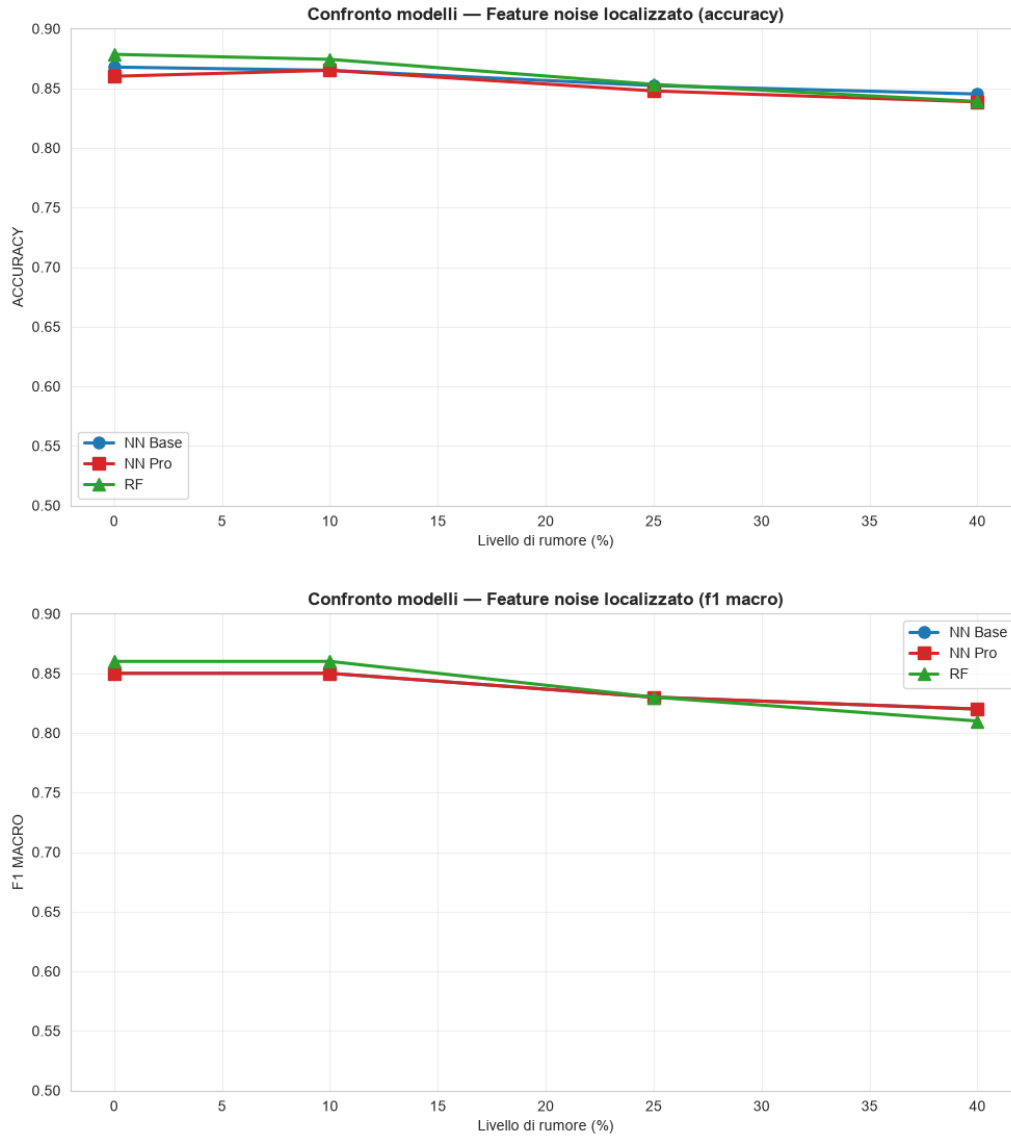


Figura 11: Feature noise localizzato: accuratezza (in alto) e F1 macro (in basso) dei tre modelli al crescere del numero di feature disturbate. Le curve restano vicine e quasi piatte, come nel feature noise globale.

Il degrado resta lieve. Anche al 40%, con cinque feature rumorose tra cui **fAlpha**, tutti i modelli restano sopra l'83% di accuratezza, in linea con il feature noise globale. Le curve di confronto (Figura 11) sono quasi piatte e i tre modelli restano vicini a ogni livello. Ritroviamo il degrado asimmetrico tra le classi già visto sugli altri disturbi sulle feature. Al 40% la recall degli adroni della rete Base scende da 0,77 a 0,67, mentre quella dei gamma sale da 0,92 a 0,94. Il *class weight* della rete Pro contiene il calo sulla classe minoritaria, con recall degli adroni a 0,75, contro lo 0,67 della Base e lo 0,64 del Random Forest.

Le curve di addestramento al 40% (Figura 12) ricalcano il feature noise globale. Nella rete Base resta un piccolo divario tra train e validation, nella rete Pro le due curve restano vicine, senza overfitting. Le matrici di confusione al 40% (Figura 13) mostrano lo stesso spostamento verso il segnale, perché parte degli adroni (riga 0) finisce tra i gamma, mentre i gamma (riga 1) restano riconosciuti.

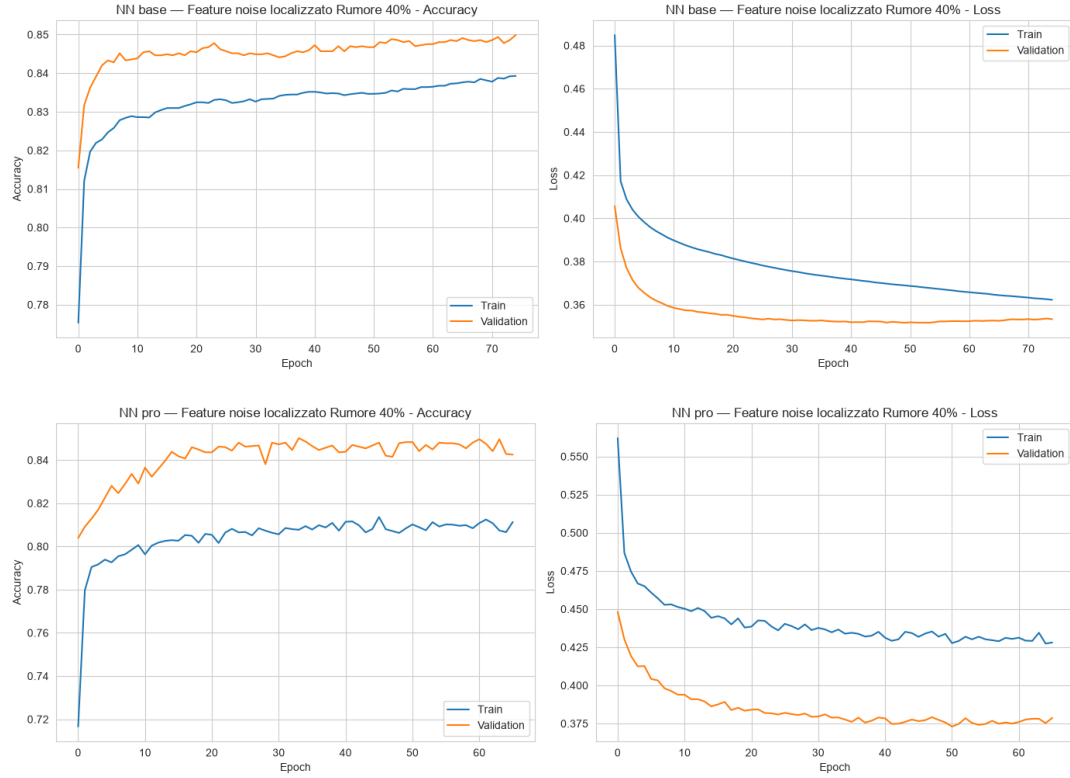


Figura 12: Feature noise localizzato al 40%: curve di addestramento della rete Base (in alto) e della rete Pro (in basso). L'andamento ricalca il feature noise globale, senza overfitting.

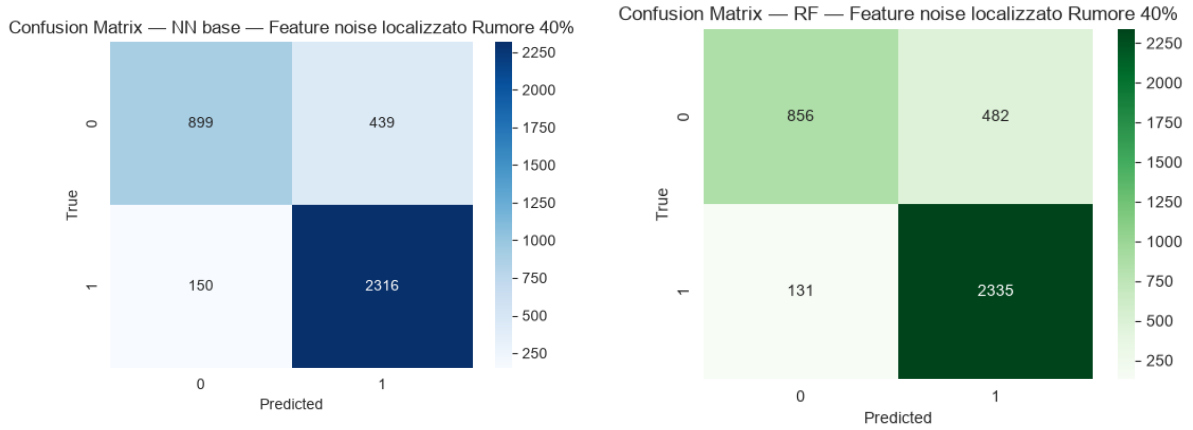


Figura 13: Feature noise localizzato al 40%: matrici di confusione della rete Base (sinistra) e del Random Forest (destra). Gli adroni (riga 0) vengono in parte classificati come gamma, mentre i gamma (riga 1) restano riconosciuti.

Il confronto più istruttivo è con il missing feature, che agisce sulle stesse feature ma azzerandole. Togliere **fAlpha** fa scendere l'accuratezza della rete Base a 0,8215 (Tabel-
la 11), mentre disturbarla con rumore la lascia a 0,8651. Il rumore *degrada* l'informazio-
ne, ma non la *cancella*, e la ridondanza tra le feature (Figura 4) basta a compensarne in
buona parte la perdita di qualità.

9.3 Label noise random

Con l'inversione casuale delle etichette il quadro cambia rispetto al feature noise. La Tabella 7 mostra che fino al 25% il degrado resta gestibile, ma al 40% due modelli su tre crollano. La rete Base scende a 0,7419 di accuratezza e 0,69 di F1 macro, il Random Forest a 0,7179. La rete Pro regge meglio, a 0,8044 di accuratezza e 0,79 di F1 macro. La Figura 14 mostra il divario. Al 40% la curva della rete Pro si stacca verso l'alto e le altre due precipitano, e lo stesso vale per l'F1 macro.

Tabella 7: Label noise random: metriche al variare del livello di rumore (test set).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8623	0.84	0.9128	0.8570	0.84	0.9200	0.8767	0.86	0.9214
25%	0.8410	0.82	0.8882	0.8378	0.83	0.9081	0.8399	0.82	0.8884
40%	0.7419	0.69	0.7513	0.8044	0.79	0.8677	0.7179	0.70	0.7673

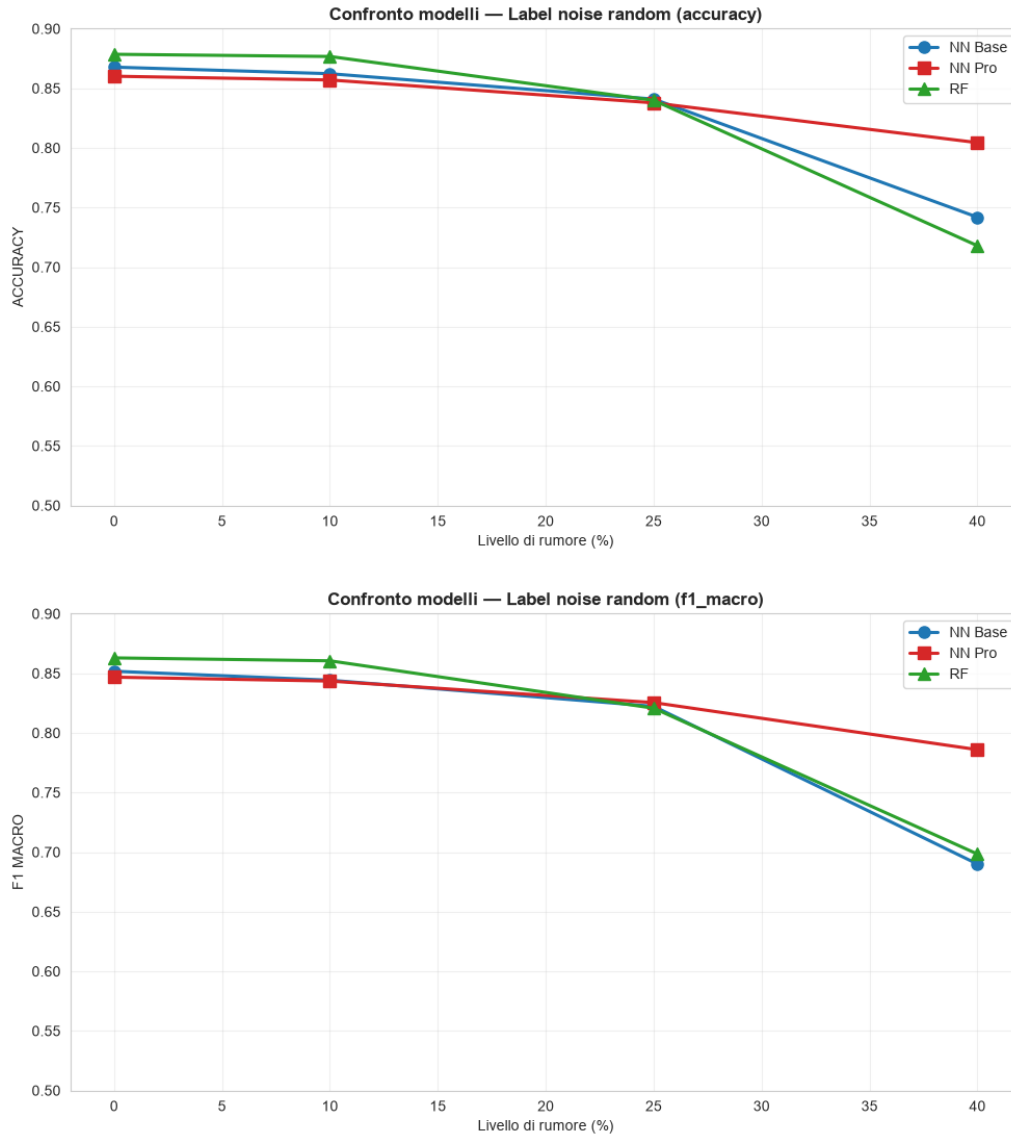


Figura 14: Label noise random: accuratezza (in alto) e F1 macro (in basso) dei tre modelli. Al 40% la rete Pro si stacca verso l'alto, mentre Base e Random Forest crollano.

Il divario si spiega con le curve di addestramento al 40%. Nella rete Base (Figura 15, a sinistra) l'accuratezza di training sale oltre l'88%, mentre l'accuratezza di validation scende e la perdita di validation cresce in modo monotono, perché la rete memorizza le etichette sbagliate. Nella rete Pro (a destra) accade l'opposto. L'accuratezza di training resta bassa, attorno al 57%, perché è misurata sulle etichette corrotte e l'early stopping ferma la rete prima che le memorizzi. L'accuratezza di validation, misurata su etichette pulite, resta alta attorno all'82% e la perdita di validation diminuisce. È il caso in cui l'arresto anticipato fa la differenza, perché impedisce alla rete di apprendere il rumore oltre il punto di generalizzazione ottimale.

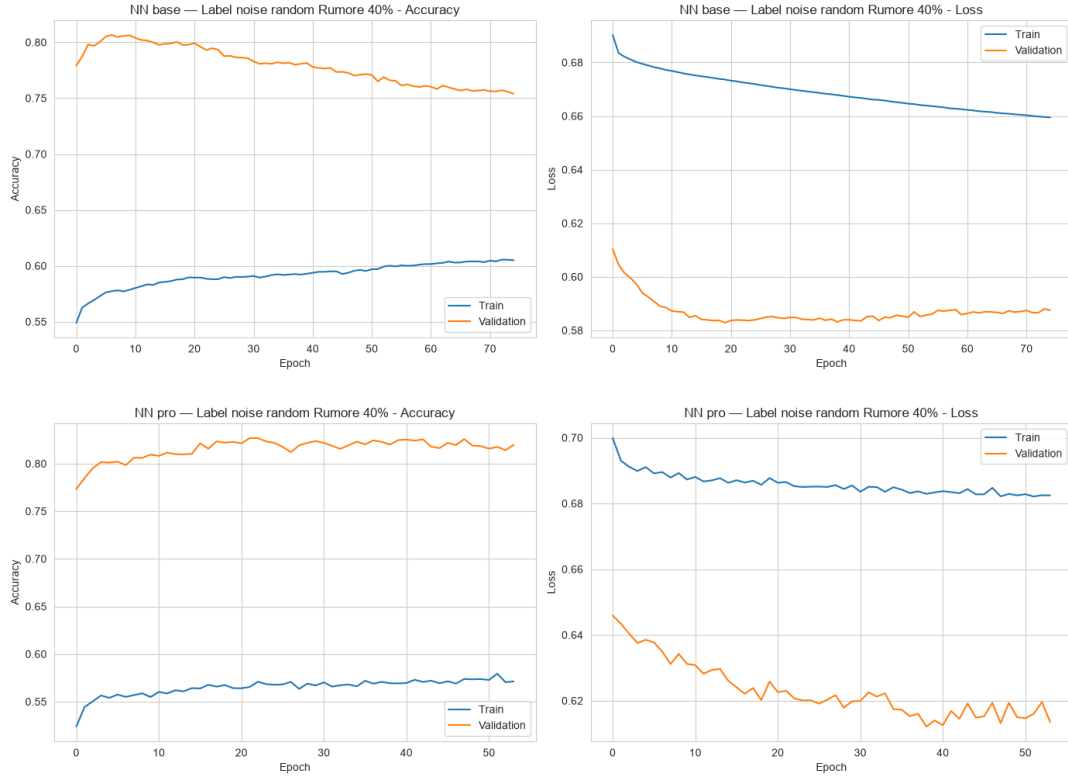


Figura 15: Label noise random al 40%: curve della rete Base (in alto) e della rete Pro (in basso). La Base memorizza le etichette corrotte (train sale, validation scende); la Pro, fermata dall'early stopping, mantiene alta la validation.

Il Random Forest non ha un meccanismo di arresto basato su un insieme pulito, quindi incorpora gran parte degli errori ed è qui il modello meno robusto. La sua matrice di confusione al 40% (Figura 16) lo conferma, perché riconosce meno bene gli adroni e gli errori si distribuiscono su entrambe le classi.

Anche qui l'impatto sulle classi è diseguale, con una sfumatura propria del rumore sulle etichette. Nella rete Base la recall degli adroni crolla da 0,77 a 0,47, mentre i gamma restano a 0,89. Come per il feature noise, paga la classe minoritaria, e di nuovo il *class weight* della rete Pro la difende (0,73 sugli adroni). Il Random Forest fa eccezione. La recall scende su entrambe le classi (adroni 0,66, gamma 0,75), perché ha assorbito le etichette corrotte in modo uniforme e l'errore non si concentra su una sola classe.



Figura 16: Label noise random al 40%: matrice di confusione del Random Forest. Gli errori aumentano su entrambe le classi: il modello ha assorbito gran parte delle etichette corrotte.

9.4 Label noise borderline

Invertire le etichette dei campioni più ambigui è il disturbo più dannoso. La Tabella 8 mostra cali già severi al 25% e gravi al 40%. Lì tutti i modelli scendono attorno al 60% di accuratezza, vicino al classificatore banale (Sezione 4), segno che gran parte della capacità discriminante è perduta. La Figura 17 mostra curve più ripide di quelle del rumore casuale.

Tabella 8: Label noise borderline: metriche al variare del livello di rumore (test set).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8252	0.80	0.8680	0.8352	0.82	0.8915	0.8383	0.81	0.8886
25%	0.7326	0.71	0.7879	0.7403	0.72	0.8265	0.7387	0.72	0.8043
40%	0.6002	0.59	0.6656	0.6354	0.62	0.7084	0.6094	0.60	0.6774

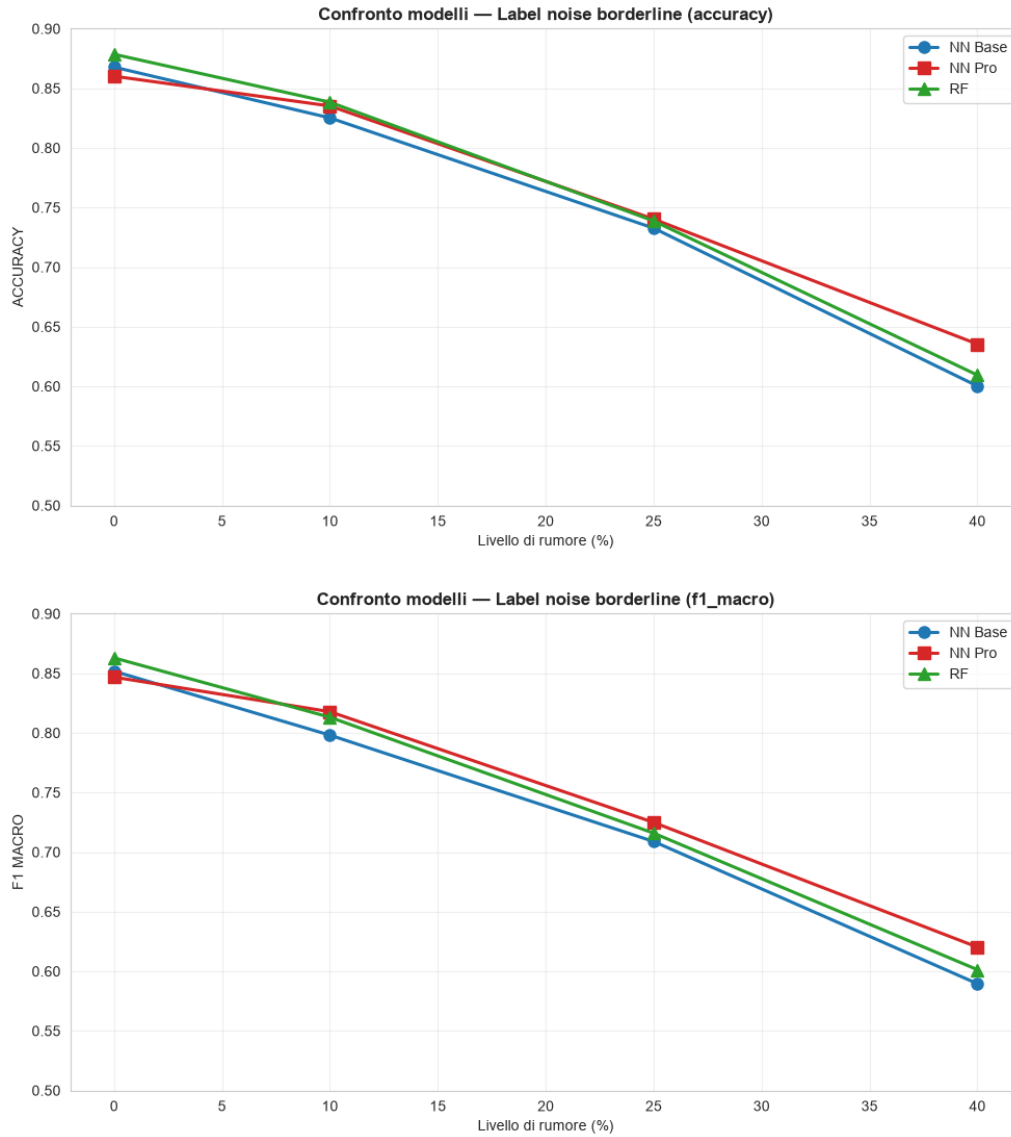


Figura 17: Label noise borderline: accuratezza (in alto) e F1 macro (in basso). Il crollo è più ripido del rumore casuale; al 40% tutti i modelli convergono attorno al 60%.

A parità di percentuale, il borderline degrada più del rumore casuale. Al 40%, ad esempio, l'accuratezza della rete Base passa da 0,7419 con rumore casuale a 0,6002 con borderline. Invertire le etichette degli esempi ambigui non aggiunge confusione uniforme, ma sposta il confine decisionale. Quei campioni, vicini alla separazione tra le classi, sono i più influenti nel determinarne la posizione. Qui il vantaggio della regolarizzazione si assottiglia. Lo mostrano le curve di addestramento al 40% (Figura 18). Anche la rete Pro, che sul rumore casuale teneva alta la validation, ora la vede scendere sotto il 65%. Quando il rumore falsifica il confine, nemmeno l'arresto anticipato recupera un'informazione corrotta nel punto più sensibile. La rete Pro conserva solo un piccolo margine, 0,6354 contro 0,6002 della Base.

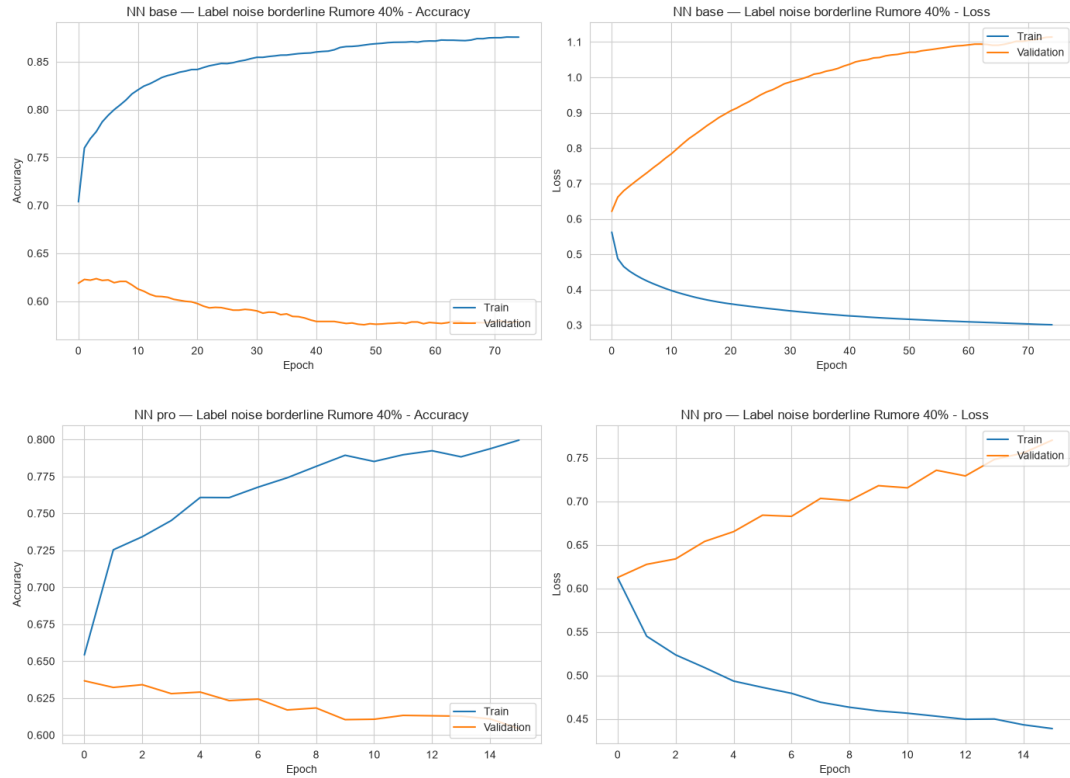


Figura 18: Label noise borderline al 40%: curve della rete Base (in alto) e della rete Pro (in basso). A differenza del rumore casuale, anche la validation della rete Pro crolla: il confine decisionale è stato spostato.

Le matrici di confusione al 40% della rete Pro e del Random Forest (Figura 19) confermano il danno. Gli errori sono diffusi su entrambe le classi e il modello fatica a separare gamma e adroni, come ci si aspetta quando il segnale di addestramento è corrotto nella regione di confine. A differenza degli altri rumori, qui il degrado è simmetrico. Al 40% la recall di adroni e gamma scende attorno a 0,60 per ogni modello (per la rete Base 0,62 e 0,59). Il modello non scivola verso la classe maggioritaria come col feature noise, ma perde potere discriminante su entrambe. Per questo il riequilibrio delle classi della rete Pro non aiuta quasi, perché il problema non è più la proporzione tra le classi, ma la posizione del confine, ormai falsificata.

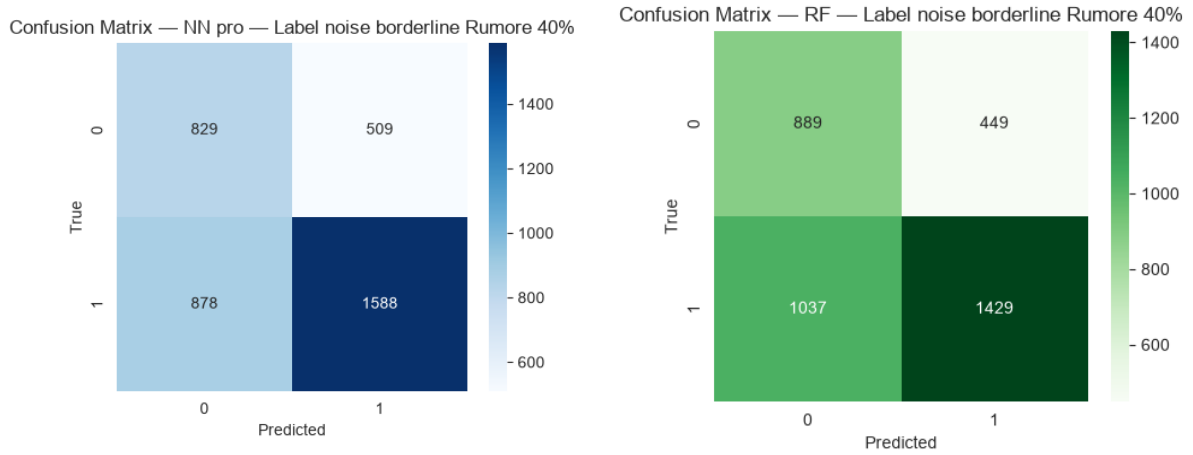


Figura 19: Label noise borderline al 40%: matrici di confusione della rete Pro (sinistra) e del Random Forest (destra). Gli errori sono diffusi su entrambe le classi.

9.5 Missing values

Con i valori mancanti il degrado è moderato e regolare, a patto di imputare. La Tabella 9 riporta i risultati con la mediana, la variante canonica. Al 40% i modelli restano tra l'82% e l'85% di accuratezza, e il Random Forest (0,8528) è il più resistente (Figura 20).

Tabella 9: Missing values (imputazione con mediana): metriche al variare del tasso di valori mancanti.

Rumore	NN Base			NN Pro			Random Forest		
	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8617	0.84	0.9172	0.8570	0.84	0.9190	0.8720	0.85	0.9208
25%	0.8425	0.83	0.9057	0.8328	0.82	0.9046	0.8578	0.84	0.9129
40%	0.8328	0.81	0.8915	0.8226	0.81	0.8937	0.8528	0.83	0.9039

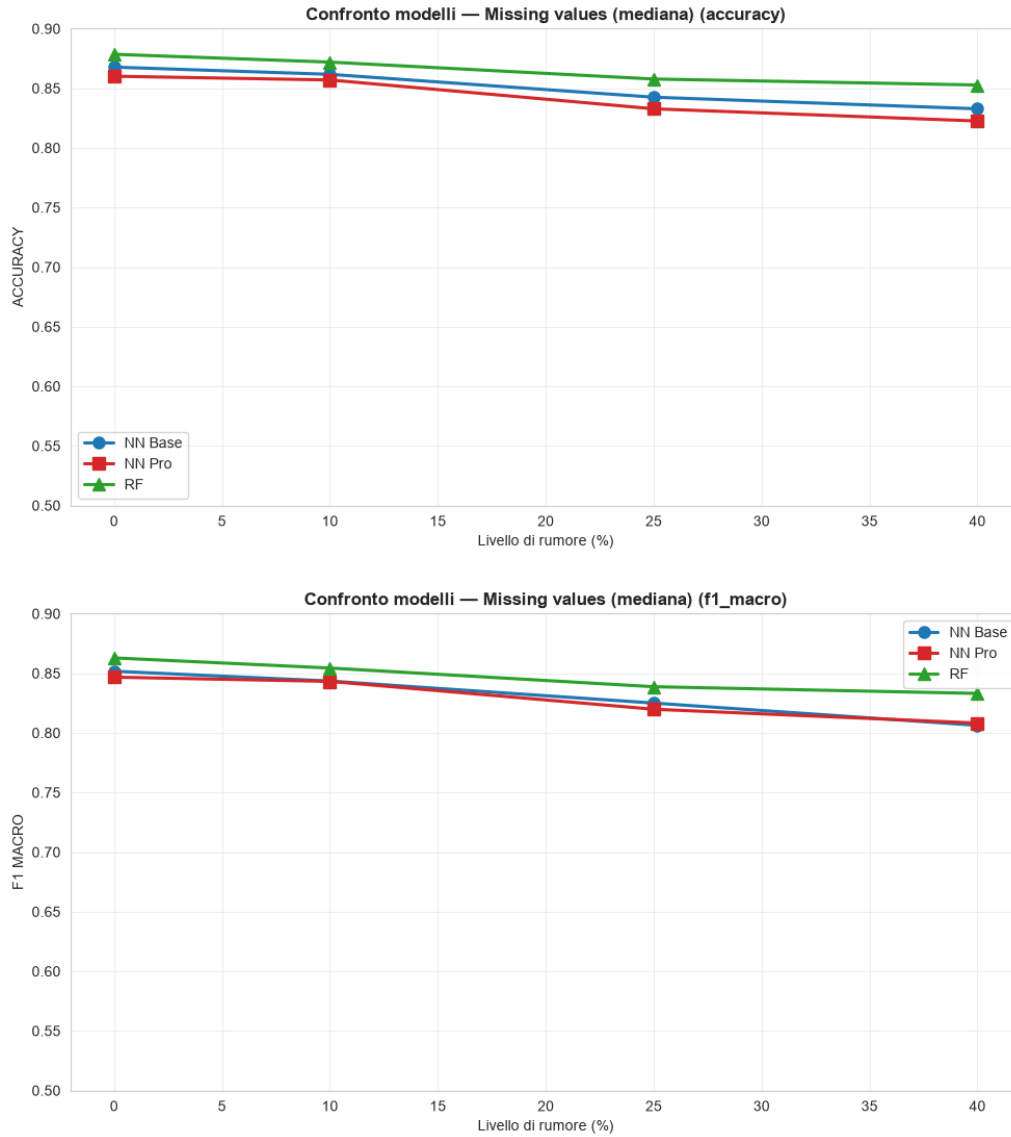


Figura 20: Missing values con imputazione mediana: accuratezza (in alto) e F1 macro (in basso). Il Random Forest mantiene un vantaggio costante a tutti i livelli.

Il confronto tra media e mediana (Figura 21) conferma quanto suggerivano le distribuzioni asimmetriche (Figura 2). La mediana è leggermente più robusta, perché non viene trascinata dalle code lunghe delle feature. Il vantaggio è modesto ma sistematico. Per la rete Base al 40% l'accuratezza passa da 0,8218 con la media a 0,8328 con la mediana, e per questo la mediana è la strategia di riferimento. Il Random Forest è più robusto per la sua natura ad alberi, perché le soglie di suddivisione risentono meno di un singolo valore imputato rispetto ai pesi continui di una rete neurale.

Ritroviamo il degrado asimmetrico già visto sulle feature. Al 40% la recall degli adroni della rete Base scende a 0,66, quella dei gamma resta a 0,93, e di nuovo il *class weight* della rete Pro contiene il calo sulla classe minoritaria (0,78 sugli adroni). Il Random Forest è il più accurato e anche il più equilibrato tra le due classi, perché l'imputazione con mediana ne preserva la capacità di riconoscere entrambi i tipi di evento.

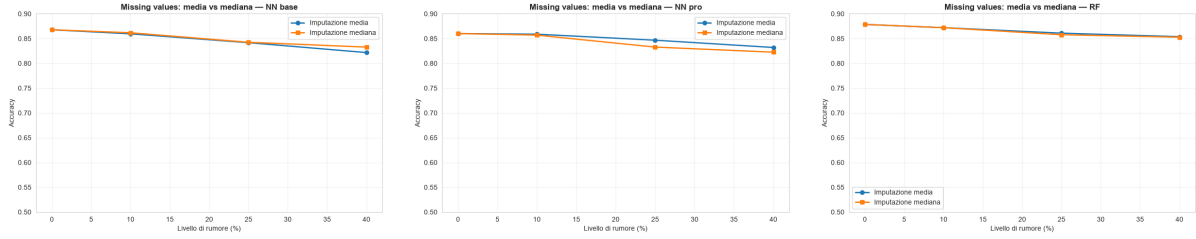


Figura 21: Missing values: confronto tra imputazione con media e con mediana per i tre modelli. La mediana mantiene un vantaggio piccolo ma costante.

Le curve di addestramento e le matrici di confusione al 40% (Figura 22) mostrano lo stesso quadro degli altri disturbi sulle feature. L'addestramento resta regolare, senza overfitting, ma gli errori si concentrano sugli adroni, in parte assorbiti dai gamma. Il Random Forest, a destra, gestisce meglio degli altri la classe minoritaria.

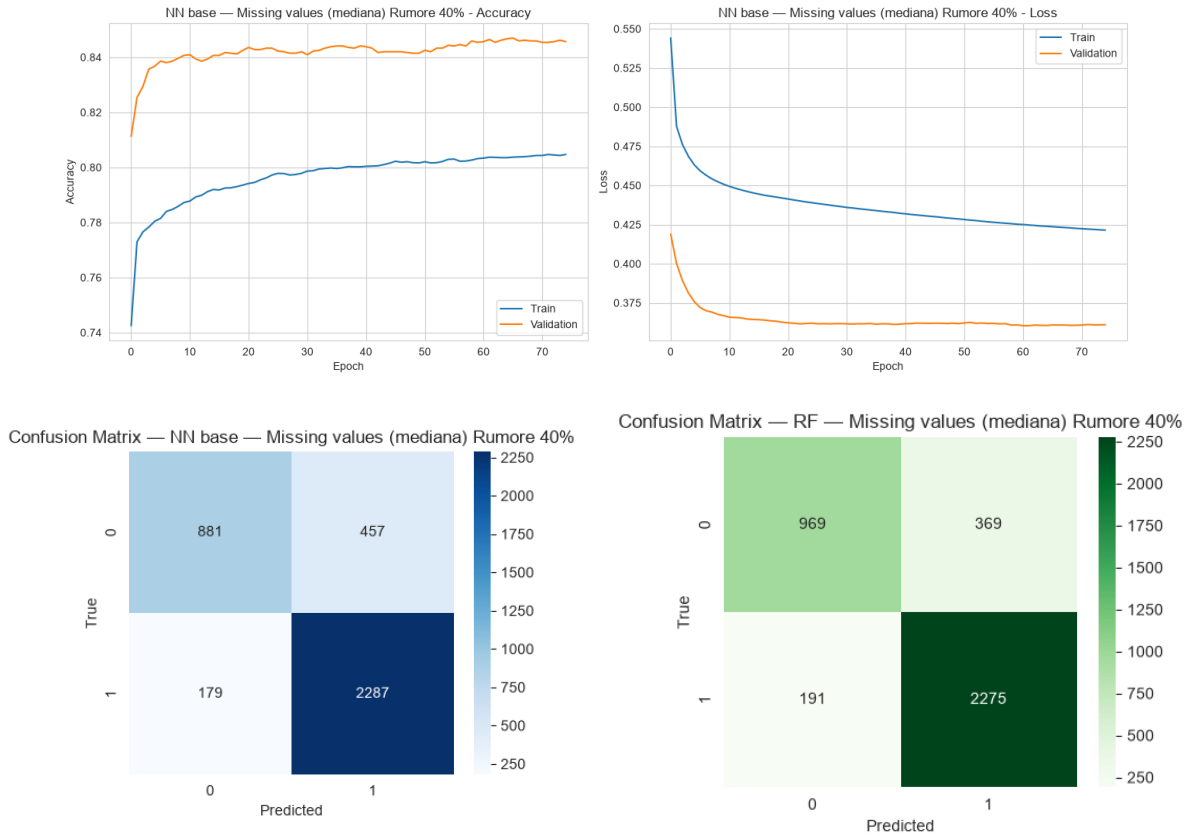


Figura 22: Missing values (mediana) al 40%: curve della rete Base (in alto), matrici di confusione della rete Base (in basso a sinistra) e del Random Forest (in basso a destra). L'addestramento è regolare; gli errori pesano soprattutto sugli adroni.

9.6 Outlier

Gli outlier artificiali sono, sorprendentemente, il disturbo meno dannoso. Con la winsorizzazione (Tabella 10) le prestazioni restano quasi invariate a ogni livello. Al 40% tutti i modelli sono ancora attorno all'85–87% di accuratezza, con un calo trascurabile. Le curve di confronto (Figura 23) sono quasi piatte sia per l'accuratezza sia per l'F1 macro.

Tabella 10: Outlier (winsorizzazione): metriche al variare della frazione di campioni anomali.

Rumore	NN Base			NN Pro			Random Forest		
	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC	Acc	$F1_m$	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8657	0.85	0.9196	0.8583	0.84	0.9220	0.8788	0.86	0.9295
25%	0.8628	0.84	0.9160	0.8441	0.83	0.9173	0.8751	0.86	0.9267
40%	0.8615	0.84	0.9116	0.8541	0.84	0.9201	0.8736	0.86	0.9250

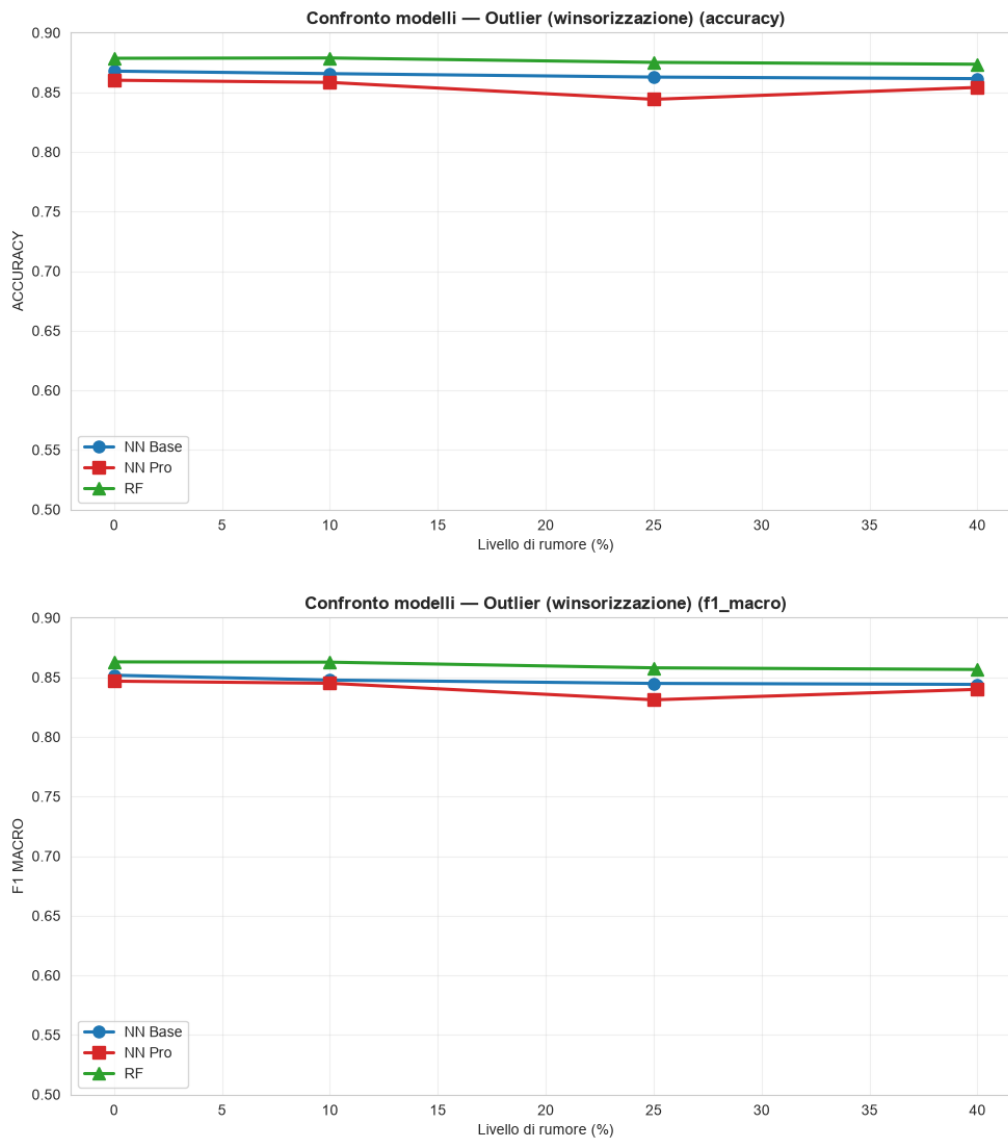


Figura 23: Outlier con winsorizzazione: accuratezza (in alto) e F1 macro (in basso). Le curve sono quasi piatte.

Le tre strategie (Figura 24) danno differenze minime ma coerenti. Anche lasciare gli outlier inalterati danneggia poco i modelli, mentre winsorizzazione e imputazione recuperano quel poco margine perso. Gli outlier entrano solo nel training e i dati sono standardizzati (Sezione 5), quindi il loro effetto sulle decisioni del modello è limitato.

Troncare le code resta comunque la scelta più prudente, perché stabilizza l'addestramento senza eliminare campioni. Adottiamo la winsorizzazione come variante canonica, perché offre il miglior compromesso tra semplicità e stabilità anche se non è indispensabile. Anche le singole classi restano stabili. Al 40% la recall di adroni e gamma resta come sui dati puliti (per la rete Base 0,75 e 0,92). Qui non c'è alcuno scivolamento verso la classe maggioritaria, perché trattati gli outlier non alterano l'equilibrio tra le classi.

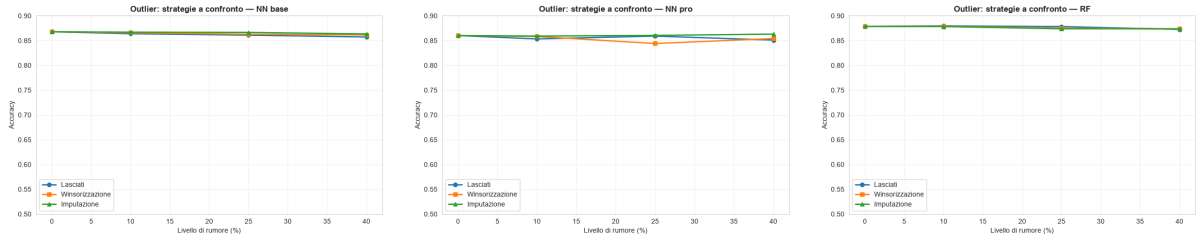


Figura 24: Outlier: confronto tra lasciarli inalterati, winsorizzazione e imputazione, per i tre modelli. Le differenze sono piccole.

Le matrici di confusione al 40% (Figura 25) sono quasi indistinguibili da quelle sui dati puliti (Figura 5), perché il numero di adroni e gamma riconosciuti resta lo stesso. La winsorizzazione neutralizza quasi del tutto l'effetto dei valori estremi.

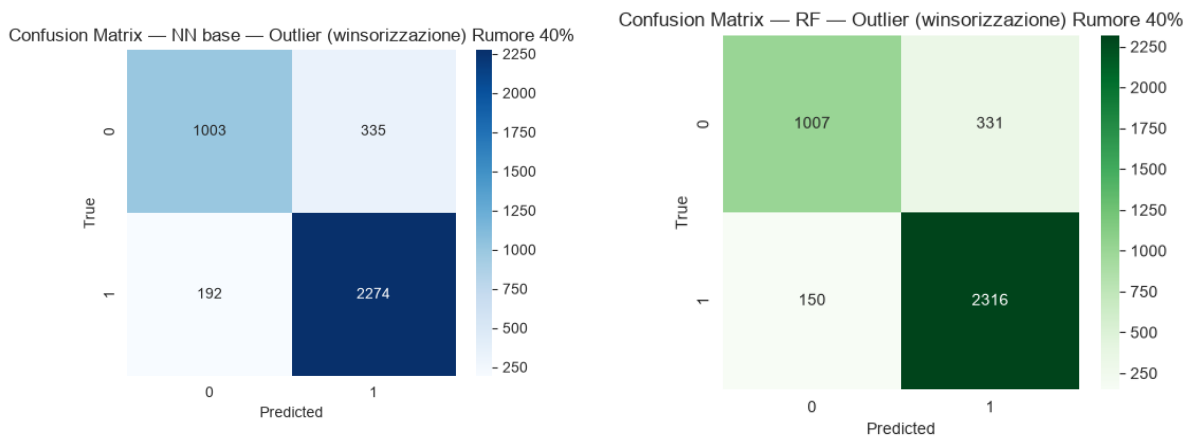


Figura 25: Outlier (winsorizzazione) al 40%: matrici di confusione della rete Base (sinistra) e del Random Forest (destra). Restano quasi identiche al caso pulito.

9.7 Missing feature (sensore guasto)

L'ultimo scenario azzerava intere feature secondo la gerarchia di importanza della Sezione 8, con `fAlpha` al 10%, le prime tre al 25% e le prime cinque al 40%. La Tabella 11 mostra un degrado severo e quasi lineare (Figura 26). Già al 10%, togliendo solo `fAlpha`, l'accuratezza scende di quattro-cinque punti, e al 40% tutti i modelli arrivano attorno al 70-74%.

Tabella 11: Missing feature: metriche rimuovendo le feature più importanti (10%→top-1, 25%→top-3, 40%→top-5).

Rumore	NN Base			NN Pro			Random Forest		
	Acc	F1 _m	AUC	Acc	F1 _m	AUC	Acc	F1 _m	AUC
Pulito	0.8678	0.85	0.9214	0.8601	0.85	0.9238	0.8785	0.86	0.9300
10%	0.8215	0.79	0.8675	0.8052	0.79	0.8716	0.8294	0.80	0.8788
25%	0.7773	0.74	0.8007	0.7574	0.73	0.7991	0.7836	0.75	0.8195
40%	0.7303	0.67	0.7413	0.6995	0.67	0.7347	0.7363	0.68	0.7494

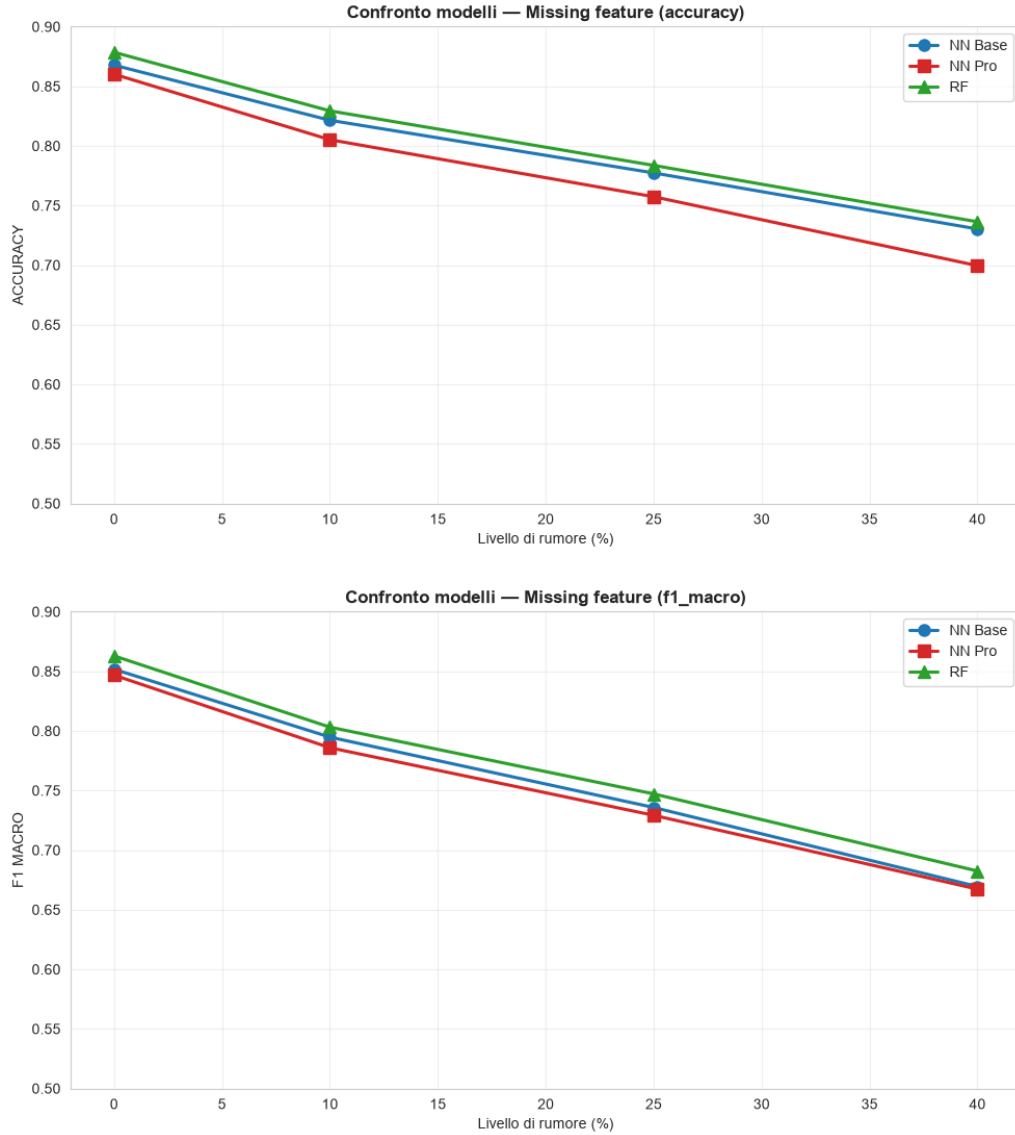


Figura 26: Missing feature: accuratezza (in alto) e F1 macro (in basso) rimuovendo le feature più importanti. Il calo è marcato fin dal primo livello, quando si elimina la sola $f\alpha$.

Il risultato conferma l'importanza di $f\alpha$ vista nei boxplot (Figura 3) e nella feature importance (Figura 7). Toglierla da sola costa più del 40% di feature noise o di outlier. A differenza degli altri disturbi, qui non c'è rumore da filtrare ma informazione

che non esiste più. f_{Alpha} è quasi scorrelata dalle altre feature (Figura 4), quindi la sua perdita non si può compensare, mentre il disturbo su una feature ridondante sì. In questo scenario la rete Pro è la meno robusta (0,6995 al 40%), perché la regolarizzazione aiuta contro il rumore, non quando manca il segnale, e la sua prudenza la penalizza un poco. Le curve e la matrice di confusione al 40% (Figura 27) mostrano un modello che, senza le feature più importanti, fatica a separare le classi, ma senza overfitting.

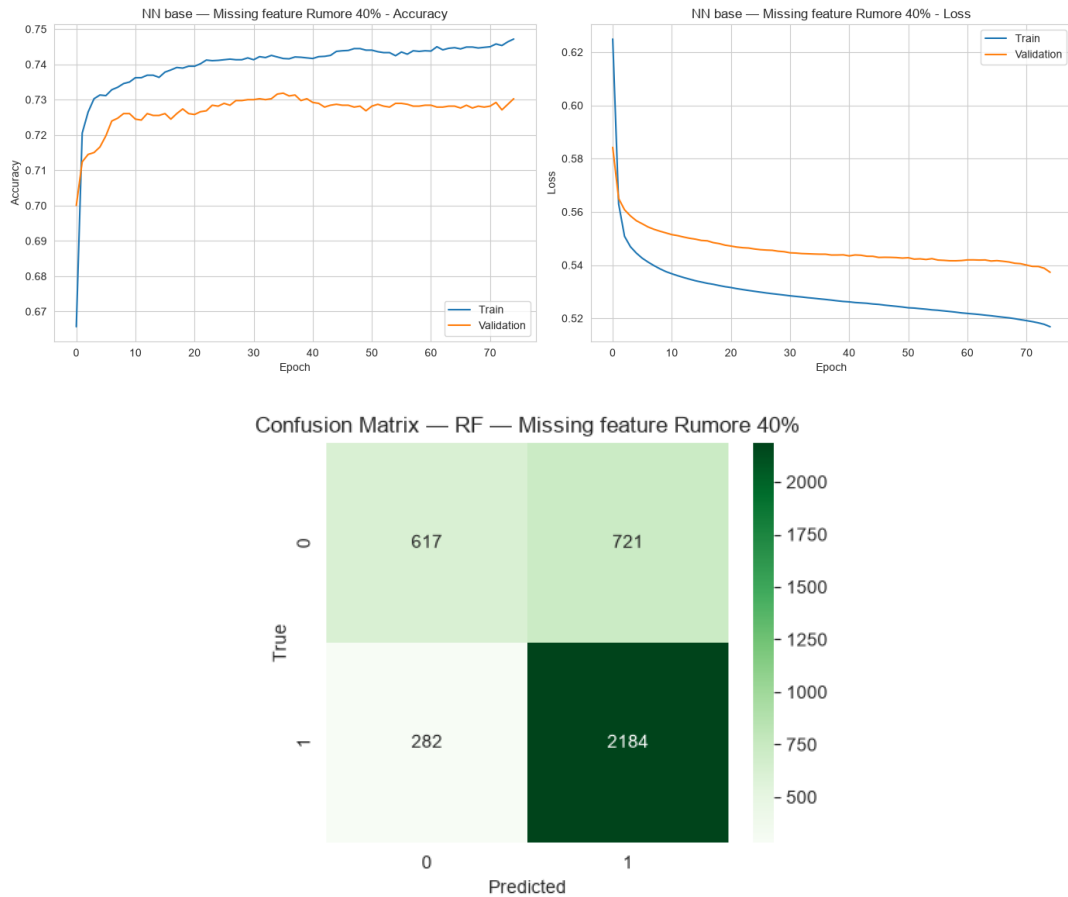


Figura 27: Missing feature al 40%: curve della rete Base (in alto) e matrice di confusione del Random Forest (in basso). Senza le feature più informative i modelli perdono potere discriminante, ma senza overfitting (curve train/validation vicine).

Qui il degrado asimmetrico tra le classi è al massimo. Togliere le feature più importanti cancella soprattutto la capacità di riconoscere gli adroni. Al 40% la recall della classe minoritaria precipita a 0,43 per la rete Base e 0,46 per il Random Forest, mentre i gamma restano sopra 0,89. Persino la rete Pro, qui la meno accurata, tiene il vantaggio sugli adroni (0,55) grazie al riequilibrio delle classi. Le matrici di confusione della rete Base (Figura 28) mostrano il deterioramento. Man mano che spariscono le feature, sempre più adroni vengono etichettati come segnale, fino a compromettere il loro riconoscimento.

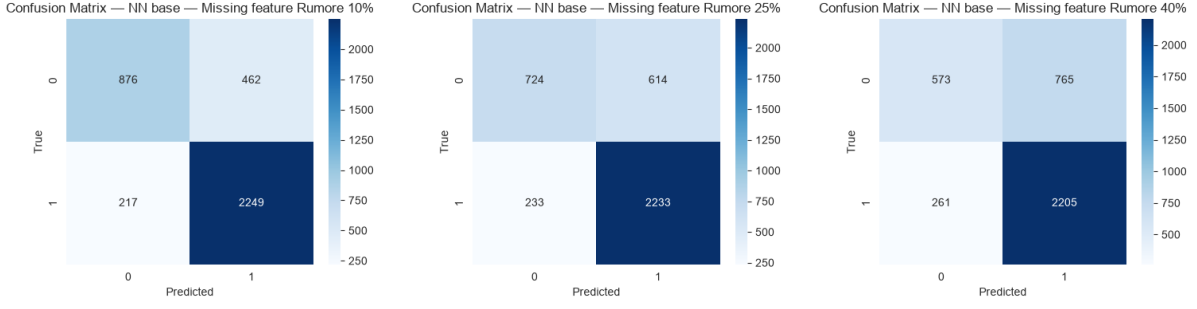


Figura 28: Missing feature, rete Base: matrici di confusione al 10%, 25% e 40% (da sinistra a destra). Rimuovendo le feature più informative il riconoscimento degli adroni (riga 0) si deteriora rapidamente.

10 Sintesi trasversale

Analizzati i sette rumori uno per uno, li mettiamo a confronto. La Figura 29 riassume, per ogni modello e su un'unica scala, la robustezza ai sette disturbi, in accuratezza e in F1 macro.

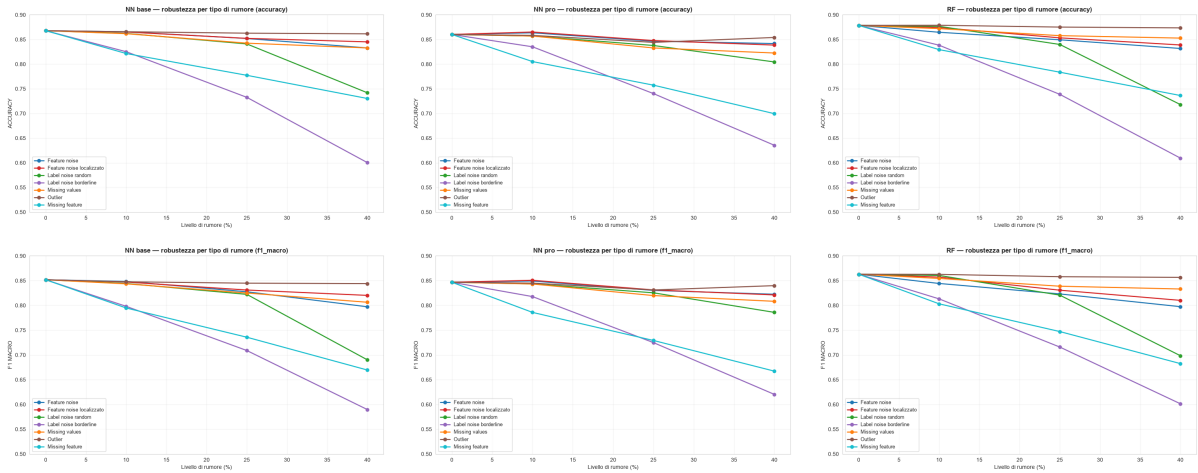


Figura 29: Robustezza per tipo di rumore: accuratezza (riga superiore) e F1 macro (riga inferiore) per rete Base, rete Pro e Random Forest (colonne da sinistra a destra). In tutti spiccano l'outlier come disturbo quasi innocuo e il label noise borderline come il più dannoso.

Emerge una gerarchia di pericolosità comune a tutti i modelli. Gli outlier, con la curva in alto quasi piatta, sono il disturbo più innocuo; seguono il feature noise, globale e localizzato, e i valori mancanti, con cali moderati; poi label noise random e missing feature, più severi; in fondo il label noise borderline, con il crollo più ripido. La heatmap dell'accuratezza nel caso peggiore (Figura 30) condensa tutto in un'immagine. Per ogni modello la colonna del borderline è la più “rossa”, quella degli outlier la più “verde”.

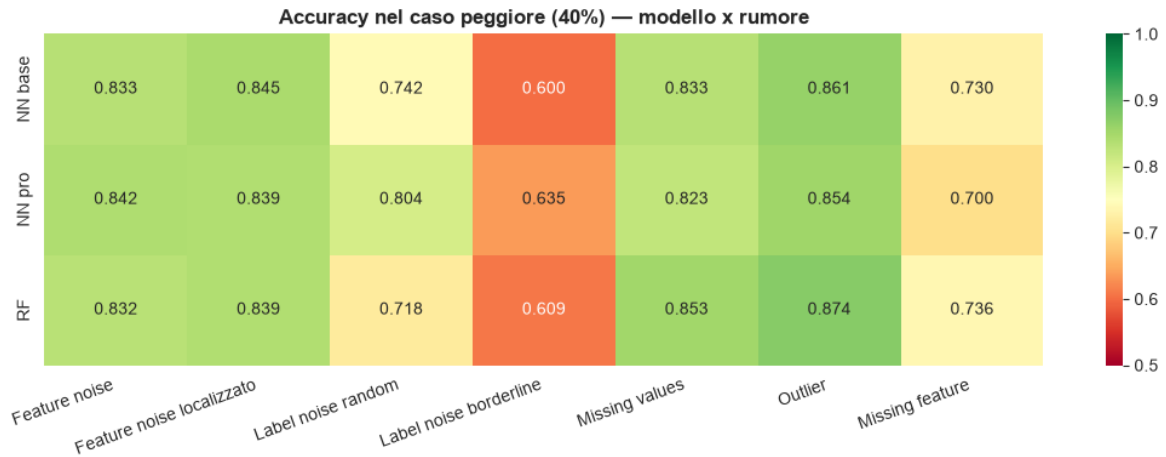


Figura 30: Accuratezza nel caso peggiore (40%) per ogni combinazione modello/rumore. Verde = prestazioni alte, rosso = basse. Il label noise borderline è il più dannoso.

Il grafico della degradazione (Figura 31) mostra invece quanto cala ogni modello passando dai dati puliti al 40% di rumore, e confronta i modelli sulla stabilità anziché sulla prestazione assoluta. Qui si vede il vantaggio della rete Pro sul label noise random, dove cala meno degli altri due, e la sua fragilità sul missing feature.

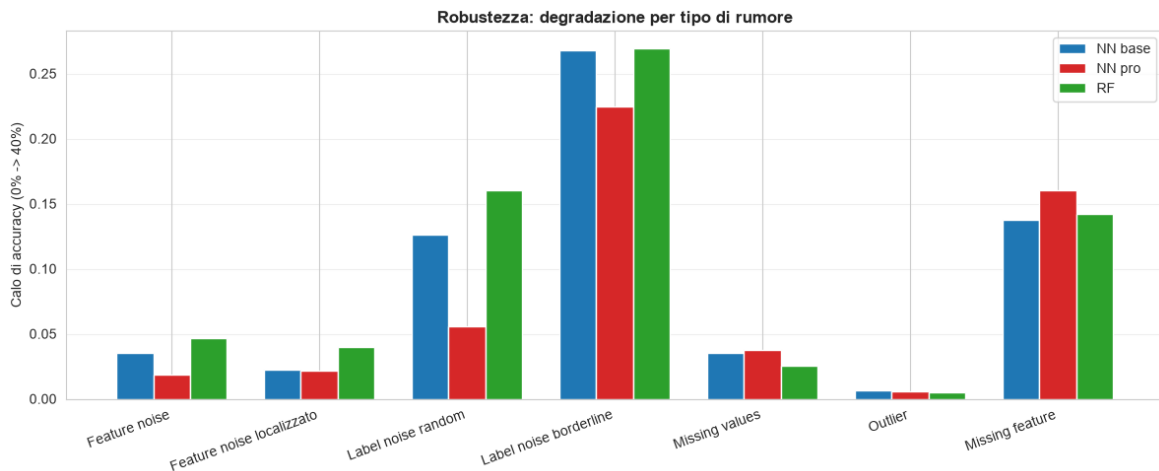


Figura 31: Calo di accuratezza (da 0% a 40%) per modello e tipo di rumore. Barre più basse indicano maggiore robustezza.

Se riassumiamo la robustezza con l'accuratezza media nel caso peggiore sui sette tipi di rumore (Figura 32), i tre modelli sono vicini. La rete Pro guida di poco con circa 0,785, poi il Random Forest con 0,780 e la rete Base con 0,778. L'esito invita alla prudenza più che a dichiarare un vincitore. La rete Pro guida grazie alla tenuta sul rumore delle etichette e delle feature, ma paga sul missing feature; il Random Forest è il più solido su valori mancanti e outlier, ma il più fragile sul label noise random. La scelta del modello dovrebbe dipendere dal tipo di corruzione attesa nei dati reali.

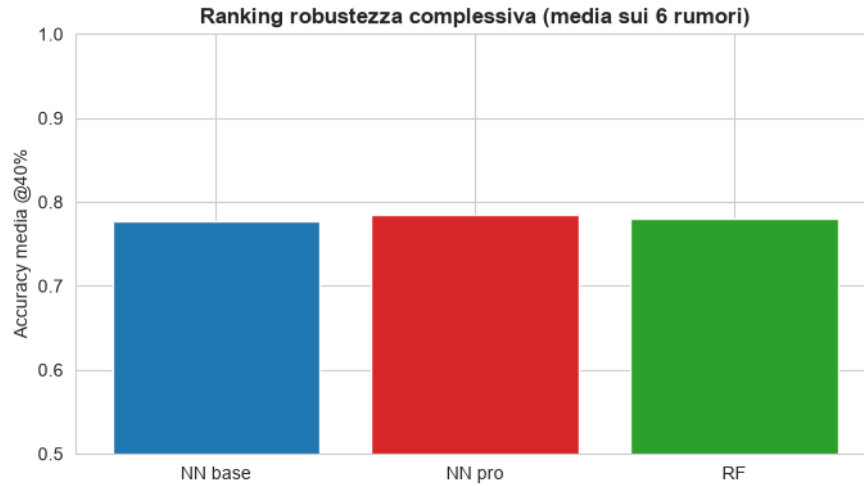


Figura 32: Robustezza complessiva: accuratezza media nel caso peggiore (40%) sui sette tipi di rumore. I tre modelli sono vicini, con un leggero vantaggio della rete Pro.

Le curve ROC tra dati puliti e 40% di feature noise (Figura 33) confermano sul ranking quanto visto sull'accuratezza. Le curve tratteggiate del rumore restano vicine a quelle continue del pulito, quindi il feature noise erode poco la capacità di ordinare i campioni. Lo scostamento maggiore è sulla rete Base, in linea con la sua tendenza ad adattarsi al rumore vista nella Sezione 9.

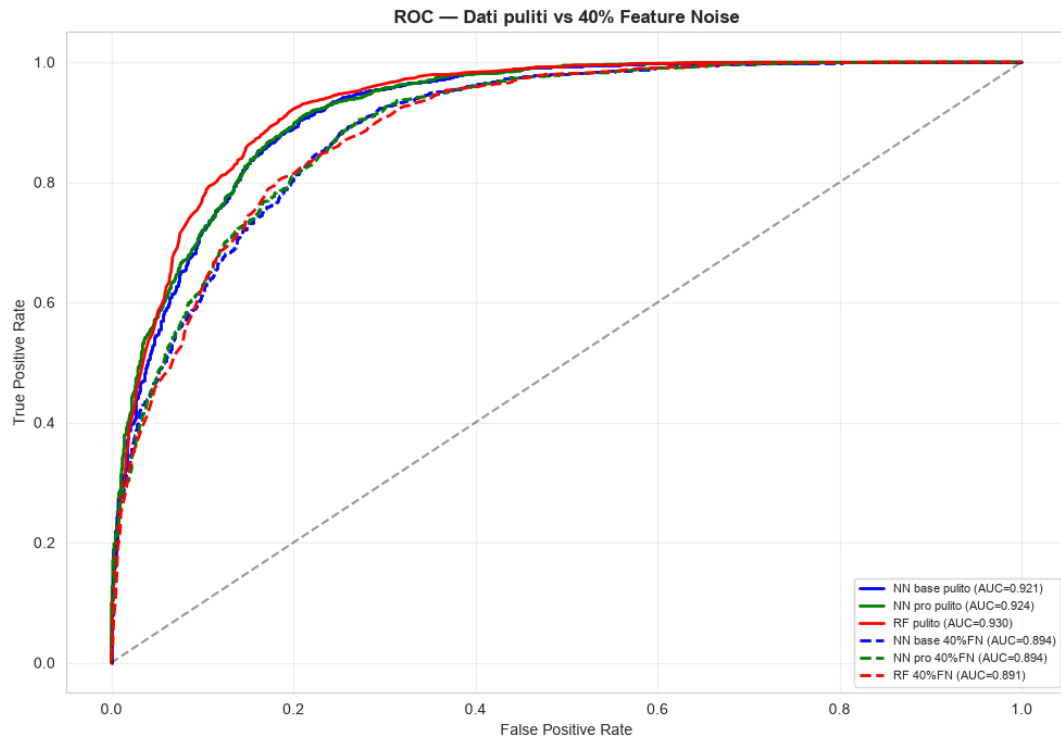


Figura 33: Curve ROC dei tre modelli su dati puliti (continue) e con 40% di feature noise (tratteggiate). Lo scostamento è contenuto: il feature noise incide poco sul potere di ranking.

10.1 Controllo multi-seed

Tutti i risultati visti finora vengono da un singolo seme casuale. Per verificare che il degrado che misuriamo sia un effetto reale del rumore e non una fluttuazione legata a quella particolare inizializzazione, abbiamo ripetuto un esperimento — il feature noise — su cinque semi diversi, che governano insieme l’iniezione del rumore e l’inizializzazione dei modelli, e ne abbiamo aggregato media e deviazione standard. Le medie ricalcano i valori a seme singolo. La rete Base passa da $0,8685 \pm 0,0027$ sui dati puliti a $0,8342 \pm 0,0031$ al 40%, il Random Forest da $0,8800 \pm 0,0013$ a $0,8285 \pm 0,0037$. La deviazione standard resta sempre sotto lo 0,007, cioè meno di un punto percentuale, mentre il calo dal pulito al 40% vale tre-quattro punti, quindi il segnale del rumore è ben distinguibile dalla variabilità tra semi. La conclusione qualitativa della Sezione 9 è quindi stabile, e non un artefatto del seme scelto.

11 Analisi non supervisionata (K-Means)

Completiamo lo studio con un esperimento non supervisionato, il K-Means con $k = 2$, che raggruppa i dati usando solo le feature, senza mai vedere le etichette. Emerge un contrasto. Il rumore sulle etichette, il più dannoso per i modelli supervisionati (Sezione 9), non ha per costruzione alcun effetto sul K-Means, mentre il rumore sulle feature ne degrada la qualità. La scelta di $k = 2$ non è imposta dalle classi note. Un’analisi preliminare al variare di k da 2 a 6, con la curva dell’inerzia (metodo del gomito) e la silhouette media, individua proprio in $k = 2$ la silhouette massima, e conferma due raggruppamenti come partizione più naturale dei dati.

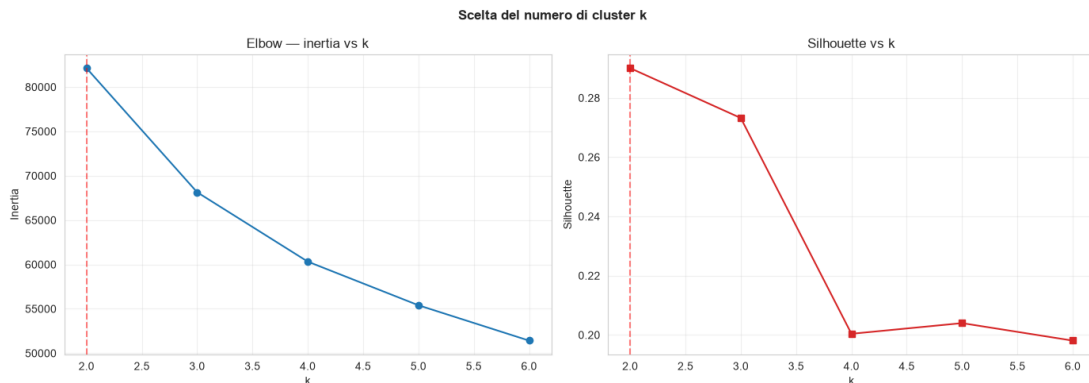


Figura 34: Scelta del numero di cluster: inerzia (metodo del gomito, a sinistra) e silhouette media (a destra) al variare di k . La silhouette è massima per $k = 2$.

I cluster geometrici del K-Means non corrispondono alle classi fisiche. Sui dati puliti otteniamo un ARI di appena 0,006 e una purezza di 0,648. La purezza coincide quasi con la proporzione della classe maggioritaria (Sezione 4), perché il clustering non distingue gamma e adroni meglio di quanto farebbe assegnando tutto alla classe più frequente. La Figura 35, che proietta i dati sulle prime due componenti principali, spiega il perché. I due cluster dividono lo spazio lungo la direzione di massima varianza, ma le classi reali sono sovrapposte e non separabili da una partizione geometrica. La struttura che separa gamma e adroni non coincide con la varianza dominante del dataset, e un algoritmo non supervisionato basato sulle distanze non la coglie.

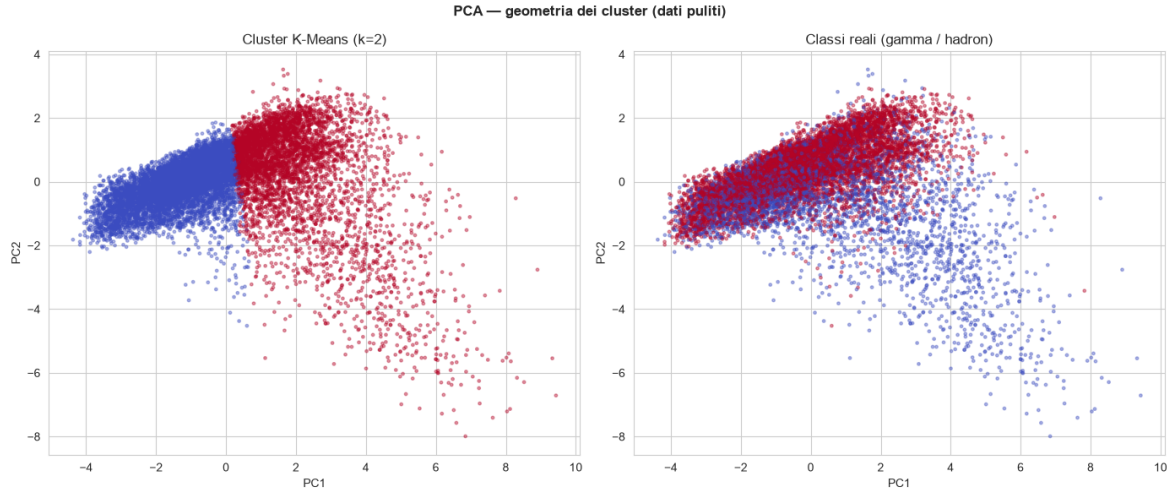


Figura 35: Proiezione PCA dei dati: a sinistra i due cluster trovati dal K-Means, a destra le classi reali. I cluster geometrici non ricalcano la separazione fisica gamma/adrone ($ARI \approx 0$).

Sul rumore i risultati confermano le attese. Con più feature noise la qualità dei cluster, misurata dalla silhouette, cala in modo progressivo, da 0,290 sui dati puliti a 0,102 con deviazione standard 2,0, perché il rumore sfuma i confini tra i raggruppamenti. La Figura 37 lo mostra. Al crescere di σ la nuvola dei punti si allarga e i due gruppi si confondono, pur restando divisi lungo la prima componente. Il label noise, invece, lascia i cluster invariati. Tra il clustering sui dati puliti e quello dopo aver invertito il 40% delle etichette l'ARI è 1,000, cioè le partizioni sono identiche. Il K-Means non usa mai le etichette, quindi corromperle non sposta i centroidi. È una conferma quasi tautologica ma utile, perché la vulnerabilità a un tipo di rumore dipende da quali informazioni il modello usa.

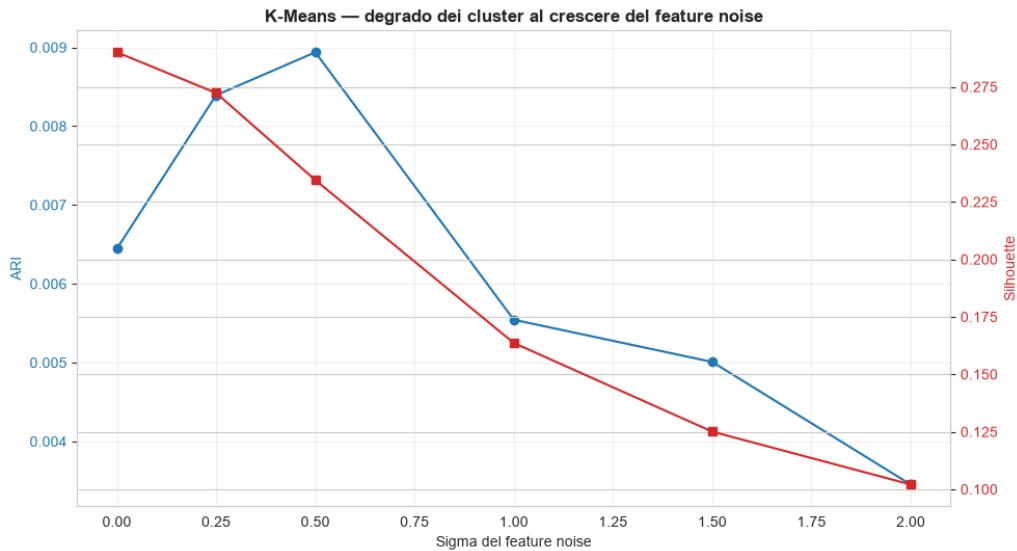


Figura 36: K-Means sotto feature noise: ARI rispetto alle classi reali (asse sinistro) e silhouette dei cluster (asse destro) al crescere di σ . La silhouette cala da 0,290 a 0,102, mentre l'ARI resta vicino a zero a ogni livello.

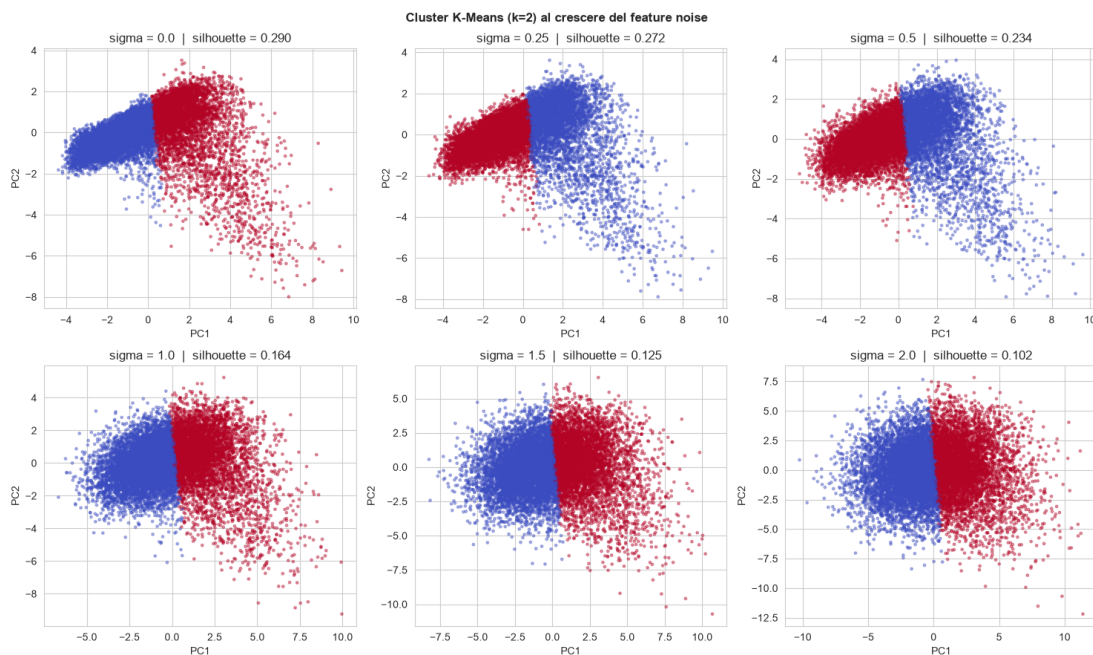


Figura 37: Cluster K-Means ($k=2$) proiettati sulle prime due componenti, ai livelli di feature noise $\sigma = 0, 0,25, 0,5, 1,0, 1,5$ e $2,0$. Al crescere di σ la nuvola si allarga e la silhouette scende da $0,290$ a $0,102$.

12 Conclusioni

Lo studio trasforma la domanda generica “quale modello è migliore” in una più precisa, “quale modello regge meglio quale tipo di corruzione”. I risultati, tutti ricavati dall’esecuzione del notebook, disegnano un quadro coerente e fatto di compromessi.

Tra i rumori c’è una gerarchia di pericolosità. Gli outlier sono i più innocui, perché con un minimo di trattamento, o anche senza, l’impatto è trascurabile. Feature noise e valori mancanti danno cali moderati e gestibili, grazie alla ridondanza tra le feature e all’imputazione. Missing feature e label noise sono i disturbi gravi, ma per ragioni opposte. Il missing feature elimina informazione insostituibile, e togliere la sola **fAlpha** costa più del massimo feature noise. Il label noise spinge le reti a memorizzare gli errori. Il caso peggiore è il label noise borderline, che corrompe i campioni vicini al confine decisionale e porta i modelli vicino al classificatore banale.

Tra i modelli, nessuno domina su tutti i fronti. La rete Pro conferma il valore della regolarizzazione, perché l’early stopping la rende più resistente al label noise random, dove blocca la memorizzazione che affonda la rete Base. La stessa prudenza non aiuta, anzi penalizza un poco, quando manca una feature. Il Random Forest è il più solido su dati puliti, valori mancanti e outlier, ma il più fragile sul label noise random. La rete Base è competitiva sui dati puliti, ma la più esposta appena il rumore cresce. In pratica la scelta del modello non va fatta in astratto, ma in base al tipo di errore atteso nei dati reali.

La lettura per classe aggiunge una sfumatura. Sotto la maggior parte dei rumori non si degradano le due classi allo stesso modo, ma quasi solo gli adroni, la minoranza, mentre i gamma restano riconosciuti. La sola accuratezza avrebbe nascosto questo calo. Per questo il riequilibrio delle classi della rete Pro, pur senza migliorare di molto l’accuratezza, è utile quando serve riconoscere la classe rara. Fa eccezione il label noise

borderline, che degrada le due classi allo stesso modo, perché ne falsifica il confine e non la proporzione.

La lezione più importante riguarda però il preprocessing, più che il modello. Strategie semplici e mirate — imputare con la mediana invece che con la media, winsorizzare gli outlier, fermare l'addestramento sulla perdita di validation — recuperano gran parte delle prestazioni perse. Nessun modello, per quanto sofisticato, recupera un'informazione cancellata, come una feature mancante, o falsificata nel punto più sensibile, come nel label borderline. La robustezza si costruisce a monte, nella qualità e nel trattamento dei dati, tanto quanto a valle nella scelta dell'algoritmo. L'esperimento non supervisionato lo ribadisce: la vulnerabilità a un rumore dipende da quali informazioni il modello usa, e ciò che è devastante per un classificatore supervisionato può essere irrilevante per un metodo che quelle informazioni non usa.